

考虑供需与电价不确定性的微网购电调度

摘要

针对微网计划购电、储能运行与日内调单的耦合关系，本文以实际总购电费用最小为目标，构建“供需预测—风险校准—库存价值控制”的统一调度框架。在共同的能量平衡和储能约束下，依次扩展可用信息与合同调整权限。

针对问题一，建立**多期线性规划模型**。利用自由弃电条件下的等价变换处理充放电互斥，联合优化购电量与储能状态。最优日购电费为 35,126.95 元，购电 59,482.70 kWh，较无储能节省 26.90%，为后续各问提供物理模型与求解基础。

针对问题二，建立**岭回归预测、分位数风险校准与历史路径库存价值模型**。以 70%误差余量控制欠购风险，利用过去 28 日配对残差估计后续缺口费用。2—12 月 334 天总费用为 13,991,392.60 元；在相同计划规则及初末库存下，库存价值控制节省 141,219.56 元。

针对问题三，建立**非对称结算滚动优化与可调单库存价值模型**。在 6、12、18 点利用新预报修订合同，按历史费用逐月选择负载修正方式，并将下次调单机会纳入库存估值。334 天总费用为 13,479,283.32 元；库存价值控制在负载修正基础上再节省 9,844.62 元。

针对问题四，在第二、三问基础上建立**波动电价下的日前与滚动调度模型**。以历史价格预测驱动计划及库存估值，以当前实际价格执行与结算，并用历史残差范围刻画价格不确定性。两分支 334 天费用分别为 14,765,492.68 元与 14,210,881.01 元，库存价值控制分别节省 115,042.37 元和 5,444.97 元。

结果表明，风险校准确定购电规模，日内更新提供合同修订依据，库存价值控制将有限电量配置给成本更高的后续缺口。四问在同一决策结构下逐层扩展，实现计划、调单与实时储能运行的协同。

关键词：微网调度；线性规划；风险校准；库存价值；滚动优化

一 问题重述

1.1 研究背景

光伏出力与小区负载在时间上存在差异。并网微网通过外部电网补充电力，并利用储能在低价或光伏富余时充电、高价或供电不足时放电，实现电量在时段间的转移。储能的容量、功率和充放电损耗共同限制了电量转移规模，因此购电计划必须与储能运行协同制定。

实际运行受到信息与交易规则的共同约束。运营者需在每天零点申报全天计划，实际负载与光伏尚未完全获知；计划不足会触发高价紧急购电，计划过量则形成额外支出。日内预报允许修订供需判断，但修改合同同样产生费用。波动电价进一步改变储能在不同时段的价值，使购电、调单与放电时机需要联合确定。

1.2 问题的提出

题目给出单日分时电价及供需曲线、全年负载与光伏实测数据、日内滚动光伏预报和波动电价四类附件[1]。微网储能额定容量为 12,000 kWh，最大充放电功率为 5,000 kW，储电量须保持在 1,200—10,800 kWh 之间，初始储电量为 6,000 kWh；充放电效率按题设取值，参数与工作假设见第 3.1 节。以十分钟为决策间隔，需依次解决以下问题。

问题一：确定性条件下的购电与储能调度。 在电价、负载逐日重复且当日光伏预测已给定时，制定全天 144 个时段的购电与充放电计划，使日初、日末储电量相等且总购电费用最小，给出指定时段购电量、全天费用及四小时储能汇总。

问题二：供需不确定条件下的日前购电。 利用决策时已掌握的历史数据，在每日零点制定购电计划；实时供电缺口按交易电价的五倍紧急补购，普通购电按计划量结算。求解 2025 年 2 月 1 日至 12 月 31 日的策略，并报告 3 月 20 日、6 月 21 日、9 月 23 日和 12 月 21 日的指定结果。

问题三：滚动预报下的计划调整。 利用 0、6、12、18 点发布的未来 24 小时整点光伏预报，调整尚未执行的购电计划；相对午夜计划，取消部分按半价、新增部分按 1.5 倍交易电价结算。制定全年滚动策略，并比较仅用午夜预报与日内更新的总费用，判断增加预报更新是否必要。

问题四：波动电价下的策略扩展。 将固定分时电价改为附件 4 给出的实时波动电价，重新计算问题二和问题三，给出两类策略的购电、充放电、紧急购电与实际费用。本文对未来价格的信息假设见第 3.1 节。

二 问题分析

2.1 问题一的分析

本问的供需与电价均已给定，各时段通过储电量递推相互关联。以购电费用最小为目标，联立能量平衡、容量、功率及日周期约束，得到多期调度模型；自由弃电条件下可用等价线性化处理充放电互斥。该物理可行域在后续各问中保持一致。

2.2 问题二的分析

本问在确定性调度中引入供需预测误差。欠购触发五倍补购，多购仍按计划付费，因此先用历史残差校准日前供需输入，再估计库存用于后续缺口的价值。前者确定应买多少电，后者决定当前应释放多少库存，最终均按实际账单评价。

2.3 问题三的分析

本问进一步允许修订尚未执行的购电合同。以当前真实储电量为起点，接入新发布的光伏预报与负载修正，在取消和新增费用约束下重新求解剩余时段。由于下次预报发布后仍可调单，库存价值估计也随合同权限更新。

2.4 问题四的分析

本问在第二、三问中同时替换计划目标、调单费用和库存价值估计中的电价输入。对尚未发生的时段使用历史价格预测，对已执行时段使用实际价格结算；其余供需预测、风险余量、物理约束和合同规则保持对应关系。重点分析价格波动下的购电与储能行为，以及日内调整的经济效果。

四问按照“共同物理约束—供需风险—日内修正—价格不确定性”递进。图 1 中每一步继承已有调度结构，只扩展可见信息或合同权限；预测为调度提供输入，线性规划生成合同计划，库存价值决定实时放电。

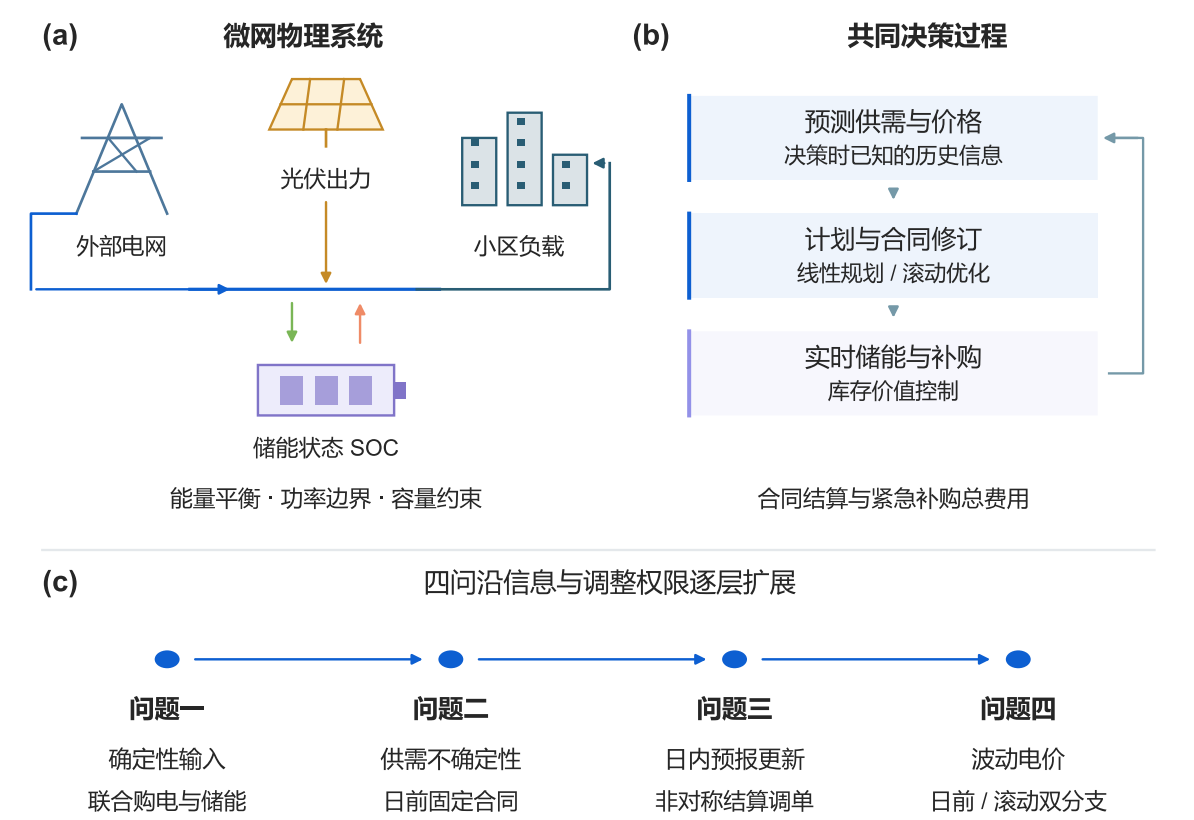


图 1 整体建模思路与四问递进关系

三 模型假设与数据处理

3.1 参数及工作假设

储能与时间参数见表 1。功率上限定义在微网母线侧，所有决策电量统一使用 kWh。将题设 90%的充放电效率解释为两侧各 90%，对应往返效率 81%；效率对运行费用的影响随第一问结果给出。

表 1 储能参数与时间离散

参数	数值	性质
额定容量	12000 kWh	题面给定
允许储电量	1200—10800 kWh	题面给定
最大充放电功率	5000 kW	题面给定
初始储电量	6000 kWh	1 月 1 日 00:00 给定
充电 / 放电效率	0.9 / 0.9	主实验解释
时段长度	1/6 h	十分钟离散

(1) 附件中的 00:10、00:20、…、24:00 依次对应 00:00—00:10、00:10—00:20、…、23:50—24:00 区间，区间内功率按常值处理。四问均保持原始记录顺序，状态定义在区间边界，功率乘 1/6 小时换算为电量。结果模板按标注的区间起点匹配输出；跨入次日的第一问区间使用周期计划，不改变当天优化的初末边界。

(2) 微网仅从外网购电，不计售电收入；无法消纳的电量允许无成本弃用，电池退化成本不纳入目标。第一、二问普通购电按计划量结算，第三、四问主动调整按保留量、取消量和新增量分项结算。

(3) 实时运行按十分钟区间内功率恒定建模，以当前可观测供需完成充放电与紧急购电；日前及日内计划仅使用决策时已经获得的信息。

(4) 第二至四问的实际储电量连续跨日。日前名义轨迹要求日末储电量不少于 6,000 kWh，以保留次日运行所需电量；实际日末状态由实时运行决定，并作为次日初始状态。

(5) 第四问将未来交易价格视为决策时未知的量，以历史数据预测；到达交易区间后观测当前价格并按其结算。题面提供全年实际价格，并未明确保证零点已知全天价格，因此本条属于信息可见性的工作假设。计划层采用点预测近似未来价格，历史残差范围描述预测误差，不据此假定未来实际价格必在某一区间内。

3.2 数据整理与时间对齐

附件 1 含 144 个时段，附件 2 的负载与光伏各含 $365 \times 144 = 52,560$ 个观测。以 2025 年完整日期和十分钟时段为索引对齐两类序列，将附件 1 混合存储的时间类型与字符串统一为分钟，核对序列为 10、20、…、1440。功率与电量按下式换算：

$$L_{d,t} = P_{d,t}^{\text{load}} \Delta t, \quad V_{d,t} = P_{d,t}^{\text{PV}} \Delta t, \quad \Delta t = \frac{1}{6} \text{ h}. \quad (1)$$

数据核查表明：附件 1 含 60 个时间类型单元格和 84 个字符串，统一转换后得到完整时间序列；附件 2 两类序列均无缺失、非有限值、负数、重复日期或完全重复日曲线。全局四分位距检验未检出超界观测，原始样本全部保留。

预处理仅规范时间格式与计量单位，无须填补、删点、缩尾或平滑。光伏序列中的 23,540 个零值按零发电记录保留。训练集标准化和预测值非负截断分别作用于特征及模型输出，不改变实际观测。

原始供需和调度轨迹保留十分钟分辨率。图中的月均值、分位范围和累计费用均由原始记录直接汇总，分别刻画季节结构、价格分布与策略收益。

四 符号说明

下文沿用表 2 的符号。单日模型省略日期下标，日内区间用 1—144，状态包含 145 个边界点；跨日计算另带日期下标。

表 2 主要符号及单位

符号	含义	单位
d, t	日期与十分钟时段	—
$\hat{L}_{d,t}, \hat{V}_{d,t}$	负载与光伏电量	kWh
$\hat{L}_{d,t}, \hat{V}_{d,t}$	日前负载与光伏预测	kWh
$g_{d,t}, e_{d,t}$	计划购电与紧急购电	kWh
$c_{d,t}, d_{d,t}$	母线侧充电与放电	kWh
$w_{d,t}$	不能利用的富余电量	kWh
$S_{d,t}$	区间起点的储电量	kWh
p_t	固定分时交易电价	元/kWh
η_c, η_d	充电与放电效率	—
z_t	充电状态二元变量	—
λ, q	岭惩罚强度与余量分位数	—
$r_{d,t}(q)$	非负净负载预测余量	kWh

日期下标与放电变量均使用字母 d ，前者仅出现在下标位置，后者作为决策变量出现，两者由位置及上下文区分。

五 第一问的模型建立与求解

5.1 供需结构与决策目标

图 2 给出单日供需曲线与分时电价。光伏集中于白天，负载贯穿全天；储能既可吸收午间富余光伏，也可利用外网峰谷价差转移购电时段。两种作用共同决定购电与充放电计划。

目标是最小化全天计划购电费：

$$\min C_1 = \sum_{t=1}^{144} p_t g_t. \quad (2)$$

以母线侧收支写能量守恒，将允许的富余供电显式记为未利用电量：

$$g_t + V_t + d_t = L_t + c_t + w_t, \quad g_t, w_t \geq 0. \quad (3)$$

充电损耗发生在进入储能时，放电损耗发生在从储能输出至母线时，因而状态方程为：

$$S_{t+1} = S_t + \eta_c c_t - \frac{d_t}{\eta_d}, \quad 1200 \leq S_t \leq 10800. \quad (4)$$

SOC 上下界施加于全部 145 个状态点，状态递推与其他时段约束对应 $t=1, \dots, 144$ 。令每时段最大母线侧充放电量 $M=5000/6$ kWh，用二元变量直接限制互斥：

$$0 \leq c_t \leq Mz_t, \quad 0 \leq d_t \leq M(1 - z_t), \quad z_t \in \{0,1\}. \quad (5)$$

第一问满足以下初始和终端条件，避免利用电池初始存量净放电造成虚假节省：

$$S_1 = S_{145} = 6000. \quad (6)$$

将上述关系集中写成一般日模型，决策变量为购电、充放电、未利用电量、储电量 and 状态变量。给定不同供需输入与初末状态条件时，可复用以下结构：

$$\left\{ \begin{array}{ll} \min_{g,c,d,w,S,z} & \sum_{t=1}^{144} p_t g_t \\ \text{s. t.} & g_t + V_t + d_t = L_t + c_t + w_t, \\ & S_{t+1} = S_t + \eta_c c_t - d_t / \eta_d, \\ & 1200 \leq S_t \leq 10800, \quad g_t, w_t \geq 0, \\ & 0 \leq c_t \leq Mz_t, \quad 0 \leq d_t \leq M(1 - z_t), \\ & z_t \in \{0,1\}, \quad S_1 = S_{145} = 6000. \end{array} \right. \quad (7)$$

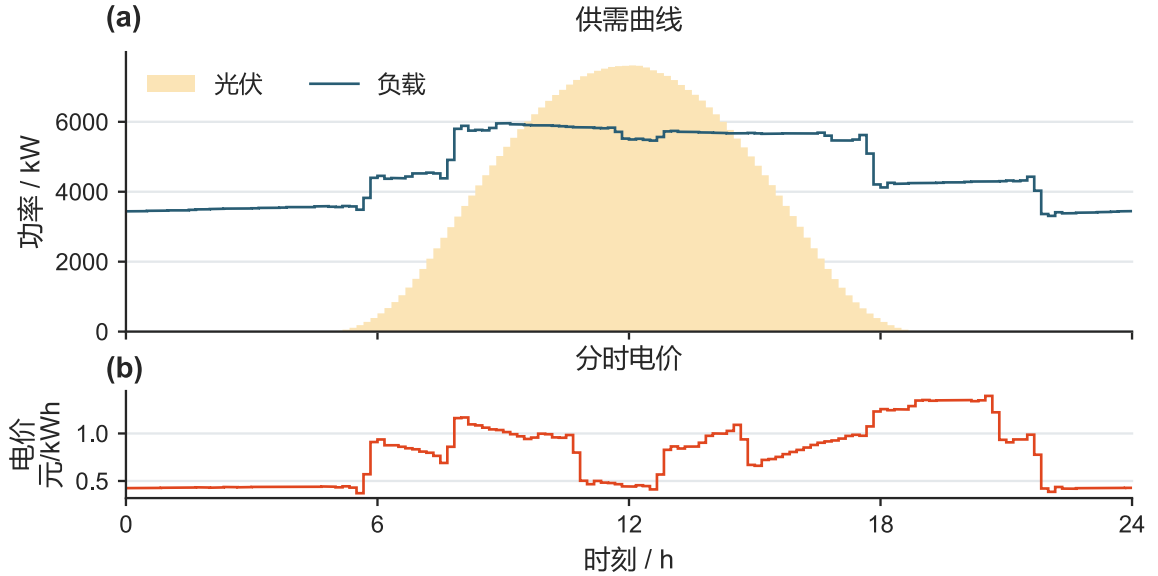


图 2 第一问供需曲线与分时电价

图 2 的每个阶梯对应一个十分钟区间，功率与电价分别使用独立坐标，光伏填充区域表示其出力过程。

为进行全年多次求解，利用自由弃电处理互斥。记 $\rho = \eta_c \eta_d$ ，对任一松弛解令 $x_t = \min\{c_t, d_t / \rho\}$ ，以 $c_t - x_t$ 、 $d_t - \rho x_t$ 和 $w_t + (1 - \rho)x_t$ 分别替代充电、放电与弃电。该变

换保持储能递推、母线平衡和购电费用，且充放电至少一项为零。因此可先用 HiGHS 求解线性规划[2]，再逐时消去循环充放电；后续各问复用相同过程。

5.2 调度结果与指定时段

计算得到全天购电量 59,482.70 kWh、费用 35,126.95 元。购电与储能轨迹见图 3，指定时段购电和四小时充放电分别见表 3、表 4。

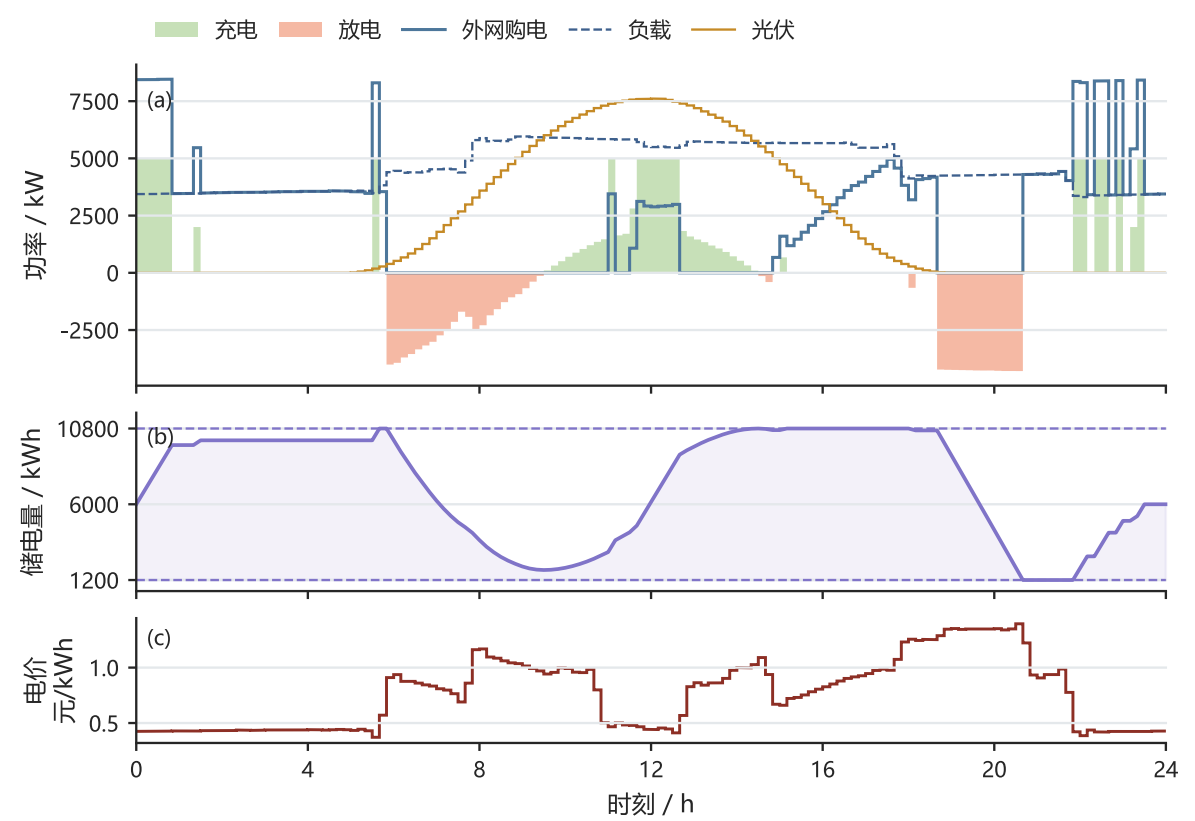


图 3 第一问电价驱动的购电与储能运行

图 3 上面板以正负方向区分充电与放电，并叠加外网购电；中面板给出边界储电量，下方面板给出分时电价。各面板共用时间轴，保留十分钟决策及 145 个状态边界。

表 3 列出指定十分钟区间的购电量及全天汇总。

表 3 第一问指定时段购电量及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.95 元		

表 4 按四小时汇总充放电量，并列出各块边界储电量。

表 4 第一问四小时充放电及边界储电量

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	6,365.84
08:00-12:00	4,787.96	1,703.00	12:00-16:00	5,286.04	91.10
16:00-20:00	0.00	5,780.13	20:00-24:00	5,333.33	2,859.87
00:00 储电量	6,000.00		24:00 储电量	6,000.00	

低价时段的外网购电同时满足负载与充电，形成集中购电峰值；高价时段由储能放电补充供电，压低外网购电量。图 3 中购电峰值与充电动作对应，反映储能的跨时段套利。表 3、表 4 分别按十分钟和四小时汇总；四小时块内可先充后放，各十分钟区间仍保持互斥。

无储能时直接按净负载缺口购电，费用为 48,052.05 元。图 4 比较无储能、实际效率和无损储能三种条件下的最优费用。实际储能净节省 12,925.10 元，降幅为 26.90%；将充放电损耗设为零后，费用进一步降至 32,377.09 元。往返效率从 64%提高到 100%时，最优费用由 38,223.65 元降至 32,377.09 元。

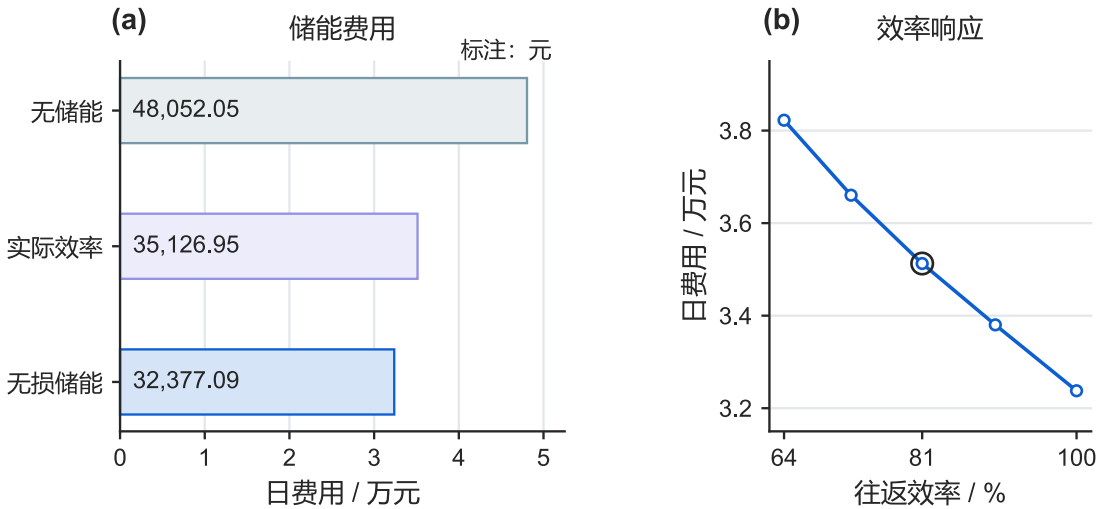


图 4 储能的净收益及效率对最优费用的影响

(a) 三种条件采用相同供需与日初日末状态，费用条形从零起算，标注为元。(b) 五组效率均重新求解，主设定为两侧各 90%、往返 81%；曲线表示效率变化后的系统最优费用。

六 第二问的预测与日前策略

6.1 训练与评价的时间顺序

采用按时间推进的训练、验证与评价划分：1 月 22—31 日选择参数与执行器，2 月 1 日至 12 月 31 日开展顺序回测。模型每七天更新，使用最近至多 60 个已结束日期。此次

模型修订属于同年度数据上的开发复核；各时点决策遵守历史可见性，评价区间不作为从未接触的外部测试集。

各候选策略从 1 月 1 日 6,000 kWh 储电量进行共同热身。首日以附件 1 作为冷启动参考，历史不足七天时使用近三日均值，随后转为日 / 周同期预测。验证期初与评价期初储电量分别为 6,244.256891 kWh 和 9,902.811288 kWh，保证各策略具有相同起点。附件 1 仅用于初始化。

6.2 供需预测

对负载或光伏电量序列 y ，同期基线取昨日与上周同日相同时间槽的均值：

$$\hat{y}_{d,t}^{\text{seasonal}} = \frac{y_{d-1,t} + y_{d-7,t}}{2}. \quad (8)$$

岭回归采用 21 个特征：昨日、前日、上周同日的同槽值，三日和七日同槽均值、七日同槽标准差、昨日全天均值和近三日全天均值共八项；前三阶日内正余弦六项；星期指示七项。日期及时间属于决策时已知的日历信息。标准化均值与尺度仅在当次训练集上估计，常量特征尺度置 1。

$$\min_{\beta, b} \sum_{i \in \mathcal{T}_d} (y_i - b - \tilde{x}_i^T \beta)^2 + \lambda \|\beta\|_2^2. \quad (9)$$

截距不惩罚，负载和光伏分别拟合。按验证 MAE 比较 $\lambda = 1, 10, 100$ ，均选 1；实现采用 NumPy 线性方程求解，与带截距岭目标一致 [3]。从 1 月 15 日开始具备拟合条件，之后每七日重新拟合。预测截断为非负；某槽过去七日光伏全零时，该槽预测置零，日出日落季节变化可能使此规则产生滞后。

评价指标采用功率单位的 MAE 与 RMSE；不使用夜间分母为零的普通 MAPE：

$$\text{MAE} = \frac{1}{n} \sum_i |P_i - \hat{P}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_i (P_i - \hat{P}_i)^2}. \quad (10)$$

6.3 净负载误差余量与名义优化

定义历史净负载误差及只包含过去信息的误差样本池：

$$\varepsilon_{j,s} = (L_{j,s} - V_{j,s}) - (\hat{L}_{j,s} - \hat{V}_{j,s}). \quad (11)$$

$$\mathcal{E}_{d,t} = \{\varepsilon_{j,s} : \max(2, d-28) \leq j < d, 1 \leq s \leq 144, |s-t| \leq 3\}. \quad (12)$$

日期下标以 1 月 1 日为 $d=1$ ；排除首日冷启动误差，最多池化 28 个已结束日期和目标槽前后三槽。跨日边界不绕回，不足 28 天时仅取已有历史。余量规则为：

$$r_{d,t}(q) = \max\{0, Q_q(\mathcal{E}_{d,t})\}. \quad (13)$$

无余量候选直接设 $r=0$ ，与“取零分位数”区分。对 $q=0.7, 0.8, 0.9$ ，分位数使用样本排序后的线性插值。将预测负载加余量作为名义需求，求解第五节同一物理模型：

$$L_t^{\text{nom}} = \hat{L}_{d,t} + r_{d,t}(q), \quad V_t^{\text{nom}} = \hat{V}_{d,t}, \quad S_{d,145}^{\text{nom}} \geq 6000. \quad (14)$$

风险校准通过历史误差分位数提高名义需求，在一月验证中按计划费与五倍紧急费之和选择余量。70%表示误差样本池的分位水平；模型属于分位数校准的确定性日前优化，其供电风险由顺序回测评价。

6.4 库存价值驱动的实时执行

零点生成计划后，普通购电费已由合同确定，执行阶段的任务是分配储能以减少后续紧急补购。若当前供给富余，先在功率与容量允许范围内充电；若出现缺口，则比较当前放电节费与保留库存的后续价值。

设当前净缺口 $n_t = L_t - V_t - g_t$ 。对富余供给，充电及弃电量为：

$$c_t = \min \left\{ \max(-n_t, 0), M, \frac{10800 - S_t}{\eta_c} \right\}, \quad w_t = \max(-n_t - c_t, 0). \quad (15)$$

为估计后续缺口，每次发布取此前至多 28 个完整日的负载与光伏配对残差，分别叠加至当前预测并截断非负，形成 K 条净缺口路径。配对保留同一天内负载、光伏误差的联系。对每条路径，以储电量 s 为状态、紧急补购费为阶段成本，反向递推后续费用：

$$J_t^{(k)}(s) = \min_{a \in \mathcal{A}_t^{(k)}(s)} \{5p_t e_t^{(k)}(a) + J_{t+1}^{(k)}(s')\}, \quad J_{145}^{(k)}(s) = 0. \quad (16)$$

可行集合 $\mathcal{A}$ 包含第一问的储能状态与功率限制，紧急购电只补负载缺口，不能用于充电。路径价值用凸分段线性函数存储[5]，在可行库存区间内计算；其均值近似实际决策的后续费用。历史路径递推用于估值，实时动作始终依据当时可见的供需和库存。

在当前真实缺口 $n_t > 0$ 时，选择下一储电状态：

$$s_{t+1}^* \in \arg \min_{z \in [\max\{S_{\min}, S_t - \min(n_t, M)/\eta_d\}, S_t]} \left\{ 5p_t [n_t - \eta_d(s_t - z)] + \frac{1}{K} \sum_{k=1}^K J_{t+1}^{(k)}(z) \right\}. \quad (17)$$

相应放电量为 $\eta_d(s_t - s_{t+1}^*)$ ，剩余缺口由紧急购电补足。求解只需比较可达区间端点与价值函数折点。当后续缺口的单位补购费更高时，模型可以保留部分库存，将电量留给更有价值的时段。实际状态按储能方程逐时更新并连续跨日：

$$S_{d+1,1} = S_{d,145}, \quad C_2 = \sum_{d,t} p_t (g_{d,t}^{\text{plan}} + 5e_{d,t}). \quad (18)$$

将“出现缺口便尽量放电”的即时平衡规则作为结果分析中的参照，以识别库存配置带来的收益。主策略使用上述价值控制；零终端附加价值由一月验证选定，名义计划仍保持日末 6,000 kWh 的下限。

七 第二问的结果分析

7.1 验证期选择与紧急购电权衡

表 5 与图 5 比较相同初始储电量下的验证费用及期末状态。随余量增加，紧急补购支出下降，普通购电支出上升；两者的折中决定余量选择。

表 5 一月份十天验证结果

预测器	余量	总费/元	紧急费/元	期末 SOC/kWh
同期	无	661,834.09	173,098.38	9,902.81
同期	70%	634,179.58	110,989.47	10,595.98
同期	80%	666,226.97	64,910.51	10,800.00
同期	90%	680,257.72	456.98	10,800.00
岭回归	无	532,888.41	61,836.07	5,924.08
岭回归	70%	509,757.47	8,765.44	6,623.17
岭回归	80%	529,846.45	4.16	7,110.16
岭回归	90%	633,664.05	0.00	9,039.38

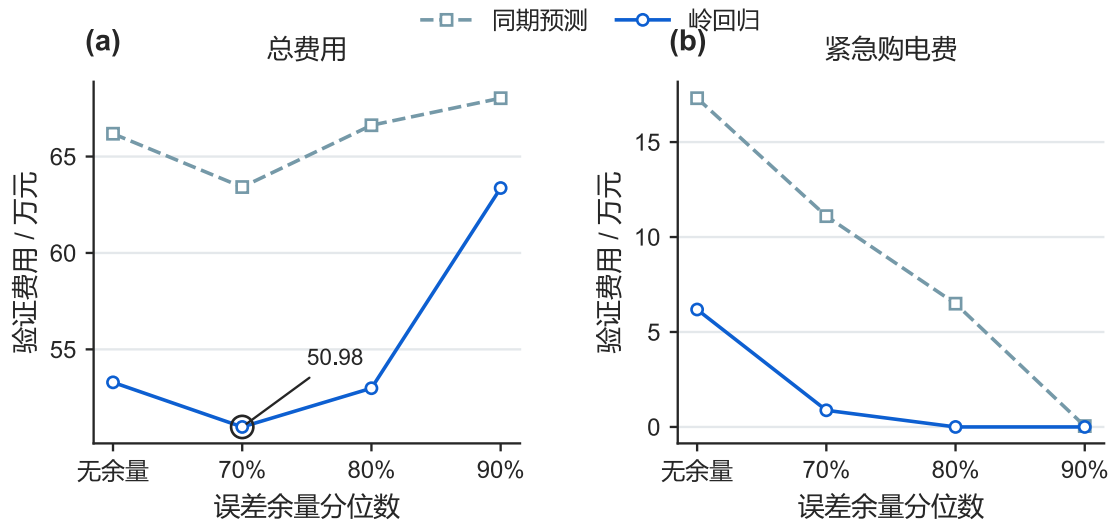


图 5 验证期风险余量与费用的权衡

图 5 分别展示候选余量的总费用与紧急购电费。增加余量通常降低紧急补购支出，但会增加普通计划支出；岭回归 70%余量的验证总费最低，为 50.98 万元。两种预测器使用相同验证日期和初始库存。

岭回归 70%余量的验证费为 509,757.47 元，其中紧急费 8,765.44 元；90%余量虽消除了验证期紧急购电，总费却升至 633,664.05 元。按总购电费用选择 70%余量，说明额外多购支出已超过进一步压缩紧急购电的收益。两方案期末储电量相差约 2,416 kWh，相关存量同时列于表 5；费用按题设结算，不计期末残值。

7.2 全期预测误差与季节变化

按表 6 比较 334 天共 48,096 个十分钟区间的功率预测误差，岭回归在负载预测上的改善大于光伏。四个指定日期的曲线对照见图 6，整年误差仍以全期统计为准。

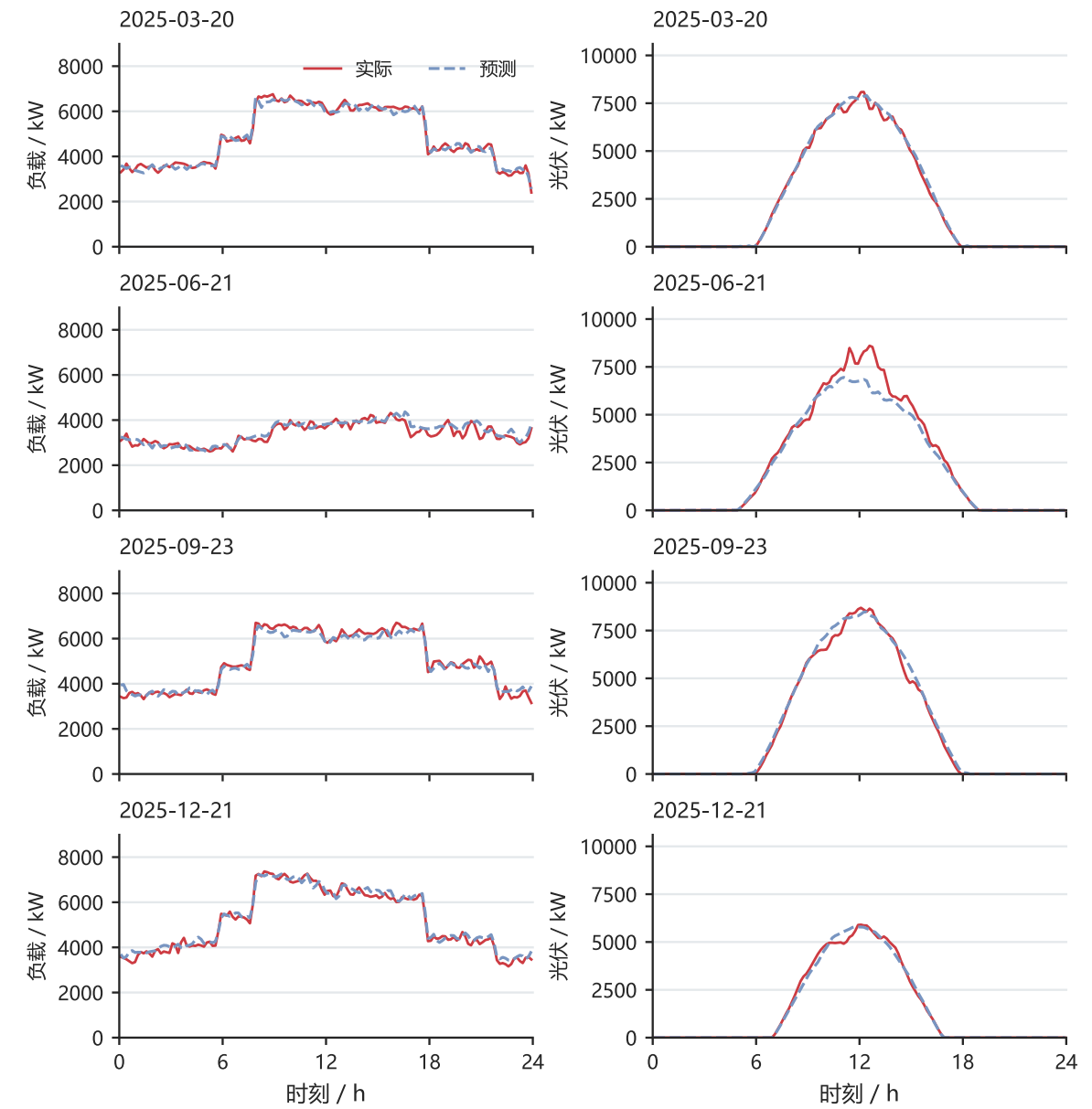


图 6 四个指定日期的负载与光伏日前预测

图 6 按日期分行、按负载与光伏分列，同一列共用纵轴尺度。实线为实际观测，虚线为岭回归预测，均保留十分钟分辨率。

表 6 第二问全期预测误差

预测器	负载 MAE	负载 RMSE	光伏 MAE	光伏 RMSE
同期	372.26	585.91	171.98	343.38
岭回归	179.86	234.59	155.28	306.74

表 6 的误差单位为 kW，光伏统计覆盖全天并包含夜间零值，表示全时段平均预测水平。

7.3 购电费用与储能配置

表 7 列出基础策略，图 7 按实际替换顺序展示从同期无余量到岭回归预测、70%风险余量及库存价值控制的费用变化。每一步均与前一步配对，直接给出各层改进的费用贡献。

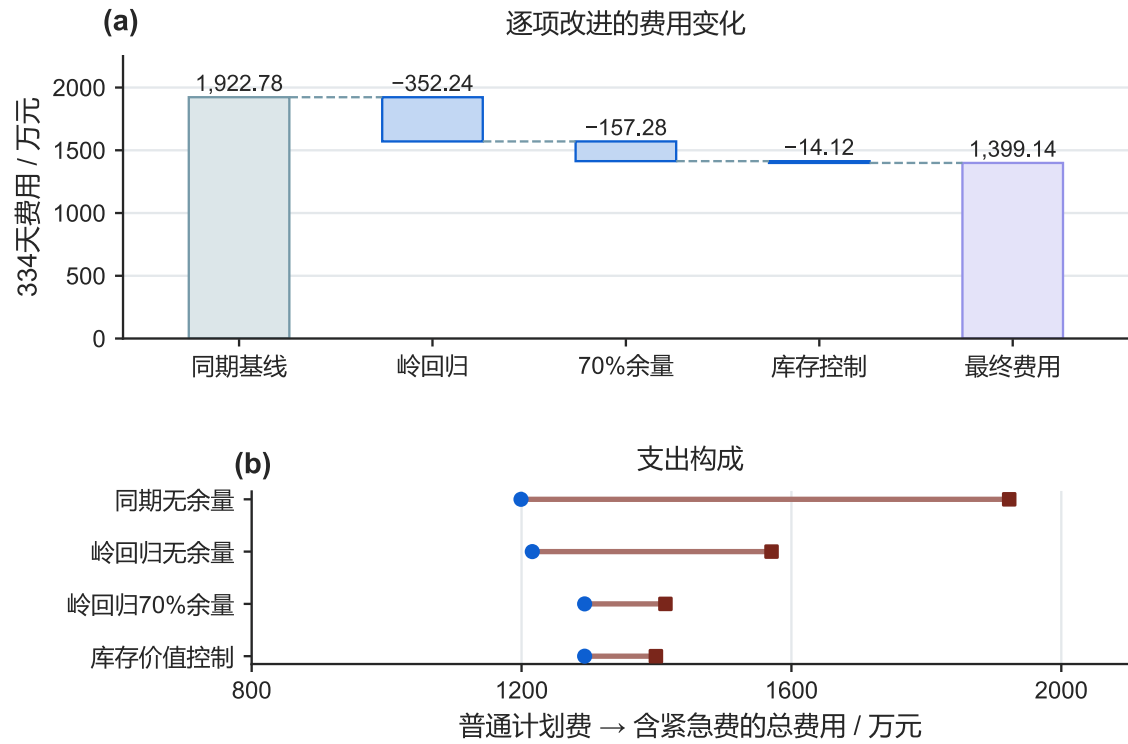


图 7 第二问逐层优化的费用变化与节费来源

表 7 第二问全期费用及期末储电量

策略	计划费/万元	紧急费/万元	总费/万元	期末 SOC/kWh
同期 无余量	1,199.19	723.59	1,922.78	5,825.99
岭回归 无余量	1,215.92	354.62	1,570.54	5,903.65
同期 70%余量	1,297.86	452.54	1,750.40	6,217.35
岭回归 70%余量	1,293.50	119.76	1,413.26	5,903.65

基础策略总费用为 14,132,612.16 元，较同期无余量降低 26.50%，较岭回归无余量降低 10.01%。岭回归两组的期末储电量同为 5,903.65 kWh，在相同预测器及初末状态下，10.01%的费用降幅直接反映风险余量的价值。

基础策略紧急购电累计 197,465.10 kWh，分布于 1,132 个十分钟区间；连续区间合并后统计为紧急购电事件。未利用电量为 2,211,229.49 kWh，由未消纳光伏与富余计划购电共同构成。风险余量通过增加部分普通购电，减少高价缺口补购。

四个指定日期的购电、充放电和紧急购电结果见附录 A。

7.4 库存价值控制的增益

在相同岭回归、70%余量、费用规则和初始储电量下，历史路径价值控制使 334 天费用由 14,132,612.16 元降至 13,991,392.60 元，节省 141,219.56 元（0.999%）。其中紧急购电费减少 141,170.39 元，而紧急电量由 197,465.10 增至 198,137.46 kWh。由此可见，收益来自补购与放电时点的重新分配：以较低价格补足当前缺口，将库存留给更高价格的后续缺口。两方案期末均为 5,903.65 kWh，库存转移的经济收益得到同状态对照支持。

八 第三问 日内预报的滚动价值

8.1 预报信息与决策时域

附件 3 含 365 天、每日 4 个版本、每版 24 个整点光伏预报，共 35,040 个值。按四行组还原 1,095 个结构性空日期后，未发现缺失、非有限数或负值。预报发布时间分别为 0、6、12、18 点；时刻 τ 的新预报只服务于 τ 之后的调度。小时预报端点之间采用分段线性函数，并在十分钟区间上积分为预测电量；发布点以最近完成区间的光伏观测衔接。

这一处理保留了预报更新带来的信息差异。每次求解均以当时真实储电量为起点，固定已经执行的购电与充放电量，重新配置其余区间。该时域推进方式与储能模型预测控制的基本框架一致[4]。

图 8 给出指定日期的实际光伏及各版预报，各预报从对应发布时间起展示。日内预报对剩余时段的光伏判断持续更新，为重配购电和储能提供信息。

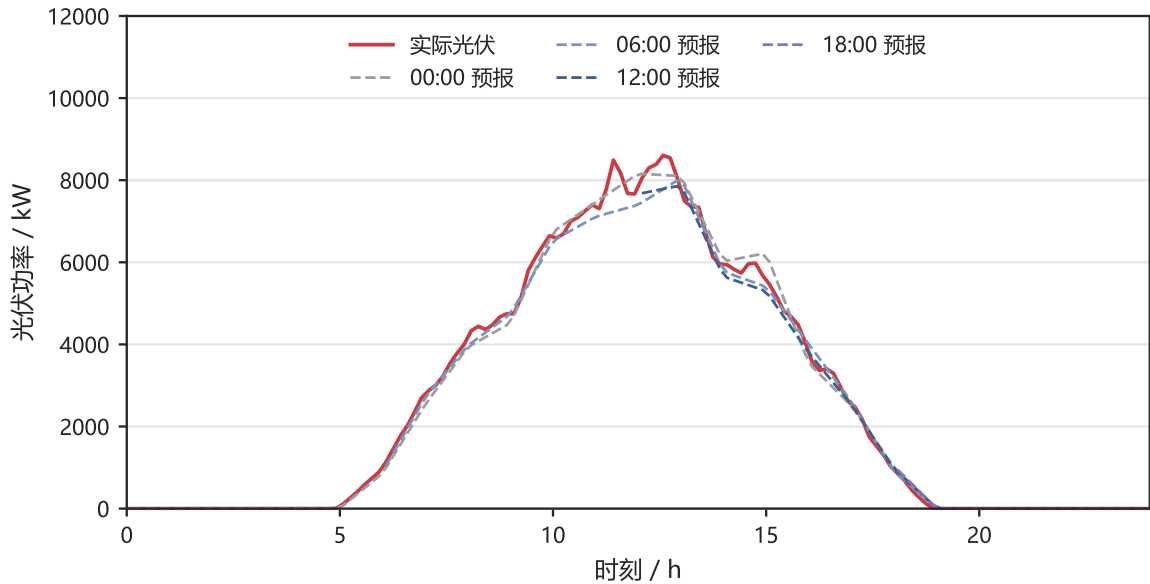


图 8 四个发布版本的光伏预报

8.2 保留量、取消量与新增量的结算

记午夜计划为 x_t ，时段执行前最后一次有效调整量为 y_t ，紧急购电为 e_t 。取消量 u_t 与新增量 v_t 分别定义为：

$$u_t = (x_t - y_t)^+, \quad v_t = (y_t - x_t)^+, \quad y_t = x_t + v_t - u_t. \quad (19)$$

保留购电按原价结算，取消部分按半价支付违约金，新增部分按 1.5 倍价格购买。因此每个区间的完整费用为：

$$F_t = p_t \min(x_t, y_t) + 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t = p_t x_t - 0.5p_t u_t + 1.5p_t v_t + 5p_t e_t. \quad (20)$$

原计划 100 kWh、调整为 80 kWh、电价 1 元/kWh 时，总费用为 $80 + 0.5 \times 20 = 90$ 元；调整为 120 kWh 时，费用为 $100 + 1.5 \times 20 = 130$ 元。前式按保留量计普通费用，后式按原计划记账，两种表达完全等价。多次预报更新均相对同一午夜计划确定偏差；中途计划用于运行与追溯，不重复累计尚未交付区间的修改费用。

8.3 滚动优化模型与风险校准

记 T_τ 为当前尚未执行的区间集合， Ω 为第一问去除首末状态等式后的物理可行域。以更新后的光伏预测、负载预测和对应误差余量作为供需输入，求解：

$$\begin{cases} \min_{y,c,d,w,S,u,v} & \sum_{t \in T_\tau} \hat{p}_{\tau,t} (1.5v_t - 0.5u_t) \\ \text{s. t.} & y_t = x_t + v_t - u_t, \quad 0 \leq u_t \leq x_t, \quad v_t \geq 0, \\ & y_t + \hat{V}_{\tau,t} + d_t = \hat{L}_{\tau,t} + r_{\tau,t} + c_t + w_t, \\ & (y, c, d, w, S) \in \Omega, \quad S_\tau = S_\tau^{\text{actual}}, \quad S_{145} \geq 6000. \end{cases} \quad (21)$$

目标函数省略了与当前决策无关的午夜合同常数项。第一至三问的 p 由附件 1 给定；第四问替换为当时可获得的价格预测。名义规划满足供电需求，基础执行按第二问即时平衡规则运行，改进执行按第 8.6 节估计后续库存价值。

风险余量取此前 28 个完整日期、相同发布版本及目标前后三个区间的净负载误差。将无余量、50%分位余量和 70%分位余量分别与 8 种更新时间组合配对，共比较 24 组方案。1 月 22—31 日验证选定 50%分位余量及 6、12、18 点更新，随后在 2—12 月评价。负载预测器和名义日末目标保持一致，以分离预报更新与调整机制的影响。

图 9 以一月验证数据比较风险余量与更新时间。左图给出 24 组候选的费用矩阵，黑框标出 50%余量、三次日内更新的选定方案；右图沿 0、6、12、18 点的顺序增加预报，展示信息更新对验证费用的影响。

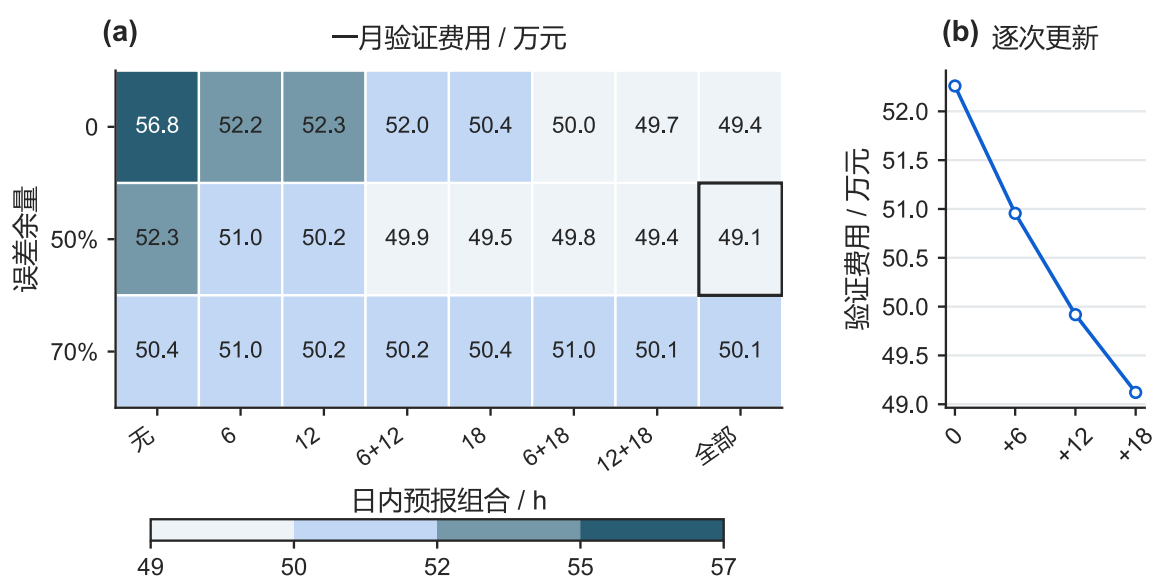


图 9 风险余量与预报更新频次的验证比较

8.4 基础滚动策略的更新时间收益

图 10 左侧按启用的预报组合列出全年节省，固定 50% 风险余量与基础执行器，分析信息更新的经济价值；右侧给出基础滚动、负载修正及库存价值控制的费用。两部分分别对应预报选择与执行策略的递进。

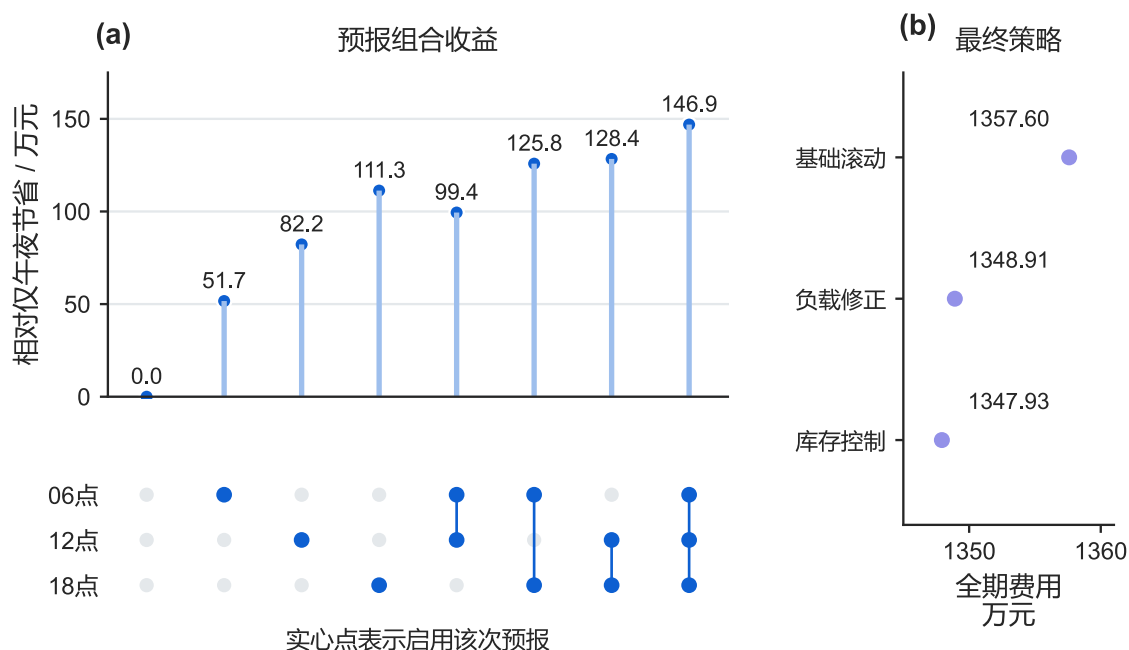


图 10 预报更新组合的全年收益与第三问最终费用

表 8 将两方案费用分解为午夜计划、净调整及紧急补购三部分。

表 8 第三问基础策略与仅午夜预报的费用分解

费用或运行量	仅午夜预报	日内滚动
原计划费/元	12,530,137.77	12,521,239.63
新增购电费/元	0.00	678,211.30
取消量净费用/元	0.00	-203,600.86
紧急购电费/元	2,514,989.16	580,126.71
总费用/元	15,045,126.93	13,575,976.78
紧急电量/kWh	407,324.31	101,019.32
期末储电量/kWh	5,903.65	5,903.65

在相同 50%余量下，三次更新将总费用由 15,045,126.93 元降至 13,575,976.78 元，净节省 1,469,150.15 元，降幅 9.76%；实际紧急购电量降至 101,019.32 kWh。新预报使部分供电缺口提前通过计划调整解决，紧急补购费用的下降足以覆盖调整支出。因此，本题有必要引入零点之外的预报。两方案初末储电量相同，节费来自运行策略改善。

8.5 面向提前时长的负载修正与月度模型选择

仅在零点预测负载，会忽略日内已观测偏差的持续性。设零点预测误差为实际负载减零点预测，以每次发布前 18 个十分钟区间的平均误差作为当前偏差 x ；历史目标小时的六个区间平均误差作为响应 y 。分别对 6、12、18 点及各未来目标小时拟合非负收缩回归：

$$\hat{\beta}_{d,v,h} = \text{clip}_{[0,1.5]} \left(\frac{\sum_{i \in H_d} x_{i,v} y_{i,v,h}}{1.2 \sum_{i \in H_d} x_{i,v}^2} \right), \quad H_d = \{\max(7, d-28), \dots, d-1\}. \quad (22)$$

其中日期从 0 编号；自第 14 日起启用，至少使用 7 个完整历史日；分母为零时系数取零。1.2 倍分母对应 20%的平方误差尺度惩罚，系数上限 1.5 限制短样本放大。修正后的预测为：

$$\hat{L}_{d,v,t}^A = \max\{0, \hat{L}_{d,t}^0 + \hat{\beta}_{d,v,h(t)} x_{d,v}\}. \quad (23)$$

同一目标小时共享修正量，零点预测不变。50%风险余量仍从该预测方案此前 28 天的历史误差计算，避免沿用旧误差分布而重复补偿。表 9 比较全期各发布时刻的负载预测误差，单位均为 kWh/十分钟。

表 9 负载自适应修正的同目标误差对照

发布时刻	基础 MAE	修正 MAE	降幅
06:00	30.26	27.96	7.61%
12:00	30.98	28.85	6.85%
18:00	31.58	27.68	12.34%

固定启用修正在 1 月 22—31 日使第三、四问滚动验证费分别增加 972.85 元和 1,149.73 元，因此不能依据后续全年收益回选一月方案。本文在验证结束后固定一项月度更新规则：2 月使用一月选定的基础预测；3 月起，每月首日分别回放此前 28 天的基础与修正方案，两者从该窗口起点的实际储电量出发，按历史真实费用选择下月方案，平局

保留基础方案。候选模型、窗口、收缩系数与费用口径在全年评价前固定，月底真实数据仅在次月选择时可见。

两类电价情景均在 3、6、7、8、9、10 月启用修正，在其余月份使用基础预测。实际储电量连续跨月传递，历史回放的末状态不覆盖实时状态。全年固定启用修正的费用另作消融结果保存，其收益不作为回选规则。该比较属于同一年度数据上的顺序开发验证，并非未接触过的外部测试集。

事后对月度验证末端库存作敏感性检查：将每 kWh 储电的价值从 0 变化至样本最高紧急电价折算值，20 次模型选择均保持不变，费用排序不依赖低估候选方案的剩余储电量。

8.6 计入下次调单机会的库存价值

直接把第 6.4 节固定合同价值函数用于滚动执行，会将后续全部缺口都按五倍紧急价计值，忽略下次发布后可新增普通合同。该方法在一月验证中增加第三问费用 4,210.28 元。为修正这一结构偏差，在当前至下次发布的区间仍按五倍价估计缺口，下一发布之后则以 1.5 倍预测价近似可新增计划的边际费用；每次发布重新构造价值函数并只执行随后 36 个时段。1.5 倍仅用于后续价值估计，实际紧急账单始终按五倍价支付。

修正方案一月费用 491,182.82 元，较即时平衡低 29.11 元；据此选定后，全年费用由 13,489,127.94 元降至 13,479,283.32 元，再省 9,844.62 元。固定合同价值法全年反而增加 149,493.07 元，验证了调单机会必须进入价值估计。该近似未显式求解未来调单、取消与库存的完整联合场景问题，其作用由同口径顺序账单检验。

为单独识别执行器贡献，两执行方案共享第 8.5 节因果负载选择信号；该信号由基础执行器的历史影子回放生成，实际库存分别连续传递，不用改进方案的未来费用重选预测模型。

九 第四问 波动电价下的购电策略

9.1 电价信息与不确定性

第二、三问采用每天重复的分时电价，本问改用附件 4 的实际价格序列。全年 365 天共 52,560 个十分钟电价，范围为 0.0076—1.7936 元/kWh。图 11 展示价格的季节变化与日内峰谷结构，同一时刻的交易成本不再逐日相同，储能的跨时段价值也随之变化。

按第 3.1 节的信息假设，决策时尚未揭示的价格以历史预测替代。以发布时刻 τ 到目标时刻 j 的预测记为 $\hat{p}_{\tau,j}$ ，实际价格分解为：

$$p_j = \hat{p}_{\tau,j} + \varepsilon_{\tau,j}. \quad (24)$$

日前分支沿用线性预测结构，以昨日、上周同槽价格，日内与星期正余弦，滞后差值、乘积及两组滞后曲线均值和标准差构成 12 维特征，标准化后用岭回归估计价格。惩罚系数取 1，1 月 15 日、22 日及后续每月首日重新拟合，训练窗口最多 60 日，所有训练目标在拟合前已经发生。预测价格下限为 0.001 元/kWh。

滚动分支采用昨日与上周同目标时段价格的均值，并在 0、6、12、18 点按最新可见历史生成预测。两个分支的价格输入规则由一月验证费用确定后固定；此处价格预测作为统一调度模型的输入模块。

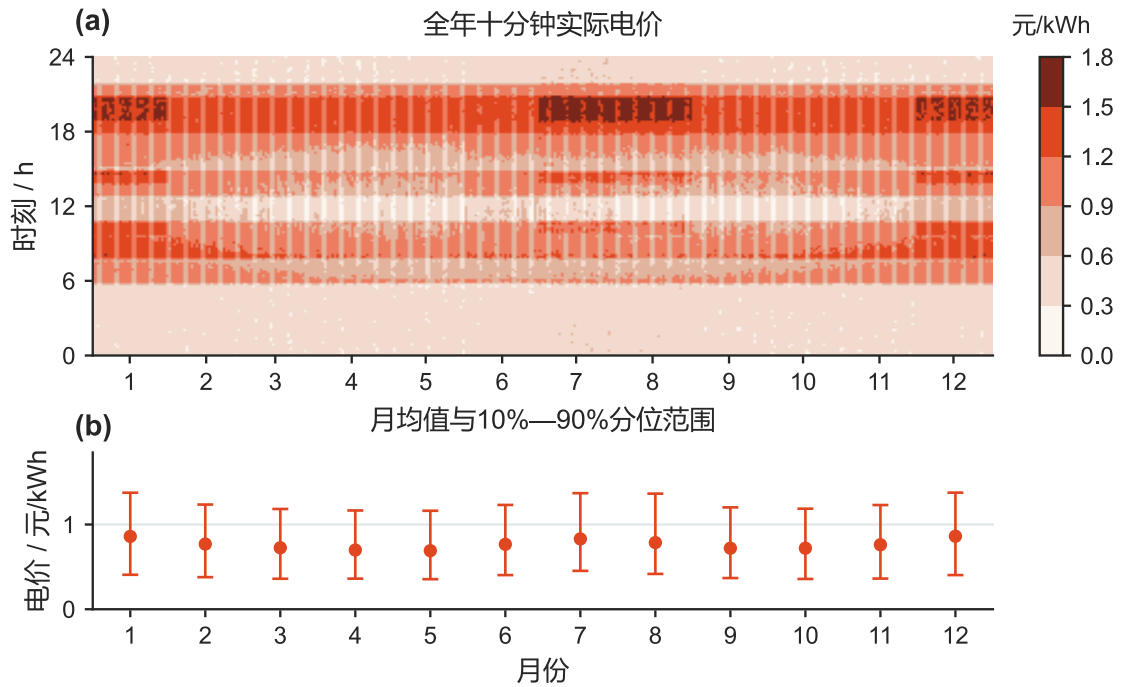


图 11 波动电价的日内结构与月度分布

(a) 全年原始十分钟价格，不截断峰谷。(b) 每月全部价格的均值与 10%—90%分位范围，表示价格分布而非均值的置信区间。

价格误差用过去 28 个完整日、相同发布时刻和目标时段的残差描述。图 12 展示四个指定日期的午夜预测及残差 10%—90%分位范围。区间端点只使用该日之前的数据；全期 48,096 个午夜预测目标的实际覆盖率为 74.36%，因此该范围用来展示预测风险，不作为未来价格的保证边界。

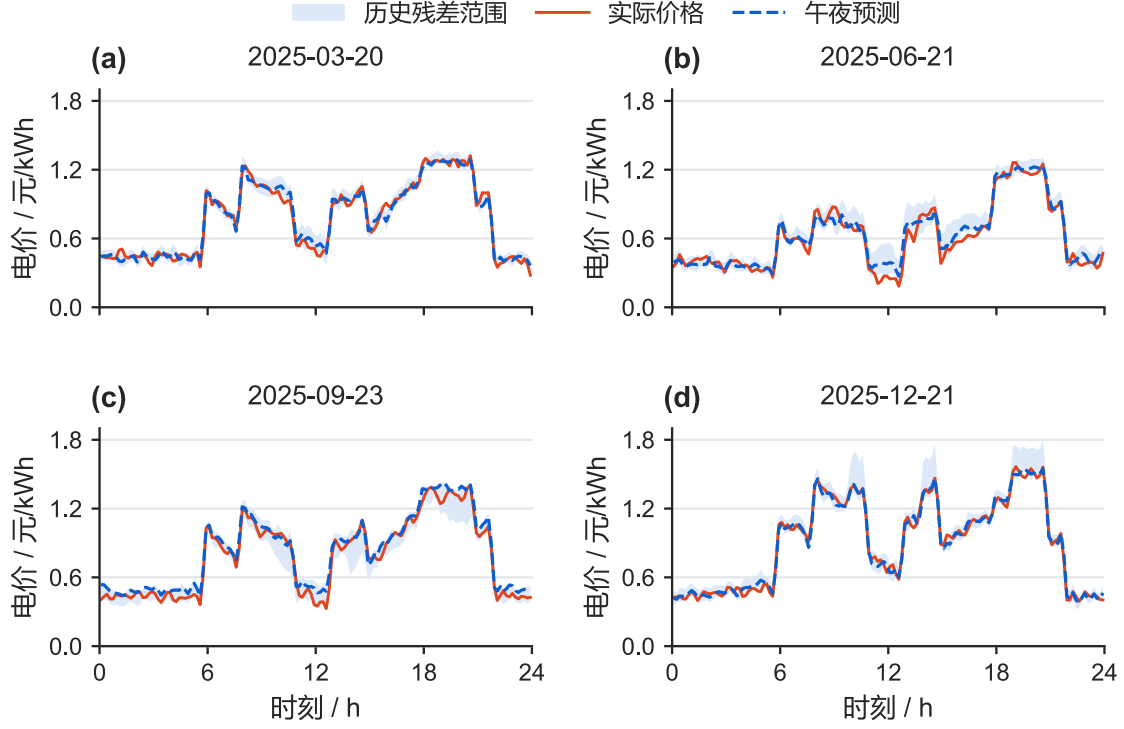


图 12 波动价格的预测偏差与历史残差范围

实线为实际价格，虚线为午夜预测；阴影为点预测加此前 28 日对应残差的 10%—90% 分位数，并将下界截至 0.001 元/kWh。四个日期由题目指定，所有面板使用相同坐标尺度。

9.2 日前与滚动模型的价格扩展

第四问继承第二问的供需预测、70%风险余量和固定合同价值控制，以及第三问的滚动光伏预报、50%风险余量、负载修正选择和可调单库存价值。新增内容是：计划目标、合同调整和库存估值中的未来价格均换为当次预测，实际结算使用交易区间的真实价格。

对于日前分支，记 $\mathcal{F}_d$ 为第二问给定当日名义供需、初始库存及日末下限后的可行域，零点计划满足：

$$g_d^0 \in \arg \min_{(g,c,d,w,S) \in \mathcal{F}_d} \sum_{t=1}^{144} \hat{p}_{d,0,t} g_t. \quad (25)$$

对于滚动分支，设 R_τ 为尚未执行的时段， k 、 a 、 b 分别为相对午夜合同保留、取消和新增的购电量，满足 $k_t + a_t = g_t^0$ 、 $g_t = k_t + b_t$ 。在第 8.2 节的合同规则下，用以下目标替换第三问固定电价目标：

$$\min \sum_{t \in R_\tau} \hat{p}_{\tau,t} (k_t + 0.5a_t + 1.5b_t). \quad (26)$$

每次滚动使用当前真实储电量、新预报和当前预测价格，过去的执行结果与午夜原合同保持不变。计划层采用预测价格的确定性等价近似，并以历史验证选择规则；供需误差继续由风险余量处理。本文未将价格误差区间设为硬约束，也未假定点预测能够消除价格风险。

实时执行沿用第 6.4 节的库存价值决策。当前缺口成本使用已揭示的实际价格 p ；未来路径费用使用 $\hat{p}_{\tau,t}$ 。日前合同锁定时，未来缺口以五倍预测价格估值；滚动分支在下次发布前采用五倍价，之后以 1.5 倍预测价格近似新增合同机会。该近似只影响库存决策，账单仍按真实合同量和交易价格计算：

$$C_{4,2} = \sum_{d,t} p_{d,t} (g_{d,t}^0 + 5e_{d,t}), \quad C_{4,3} = \sum_{d,t} p_{d,t} (k_{d,t} + 0.5a_{d,t} + 1.5b_{d,t} + 5e_{d,t}). \quad (27)$$

因此，价格变化同时影响计划购电时段、日内调单与当前库存保留。图 13 按题目指定的四个日期分列，对齐价格、储电量及累计紧急购电的日内轨迹，展示两套策略在不同供需条件下的响应。

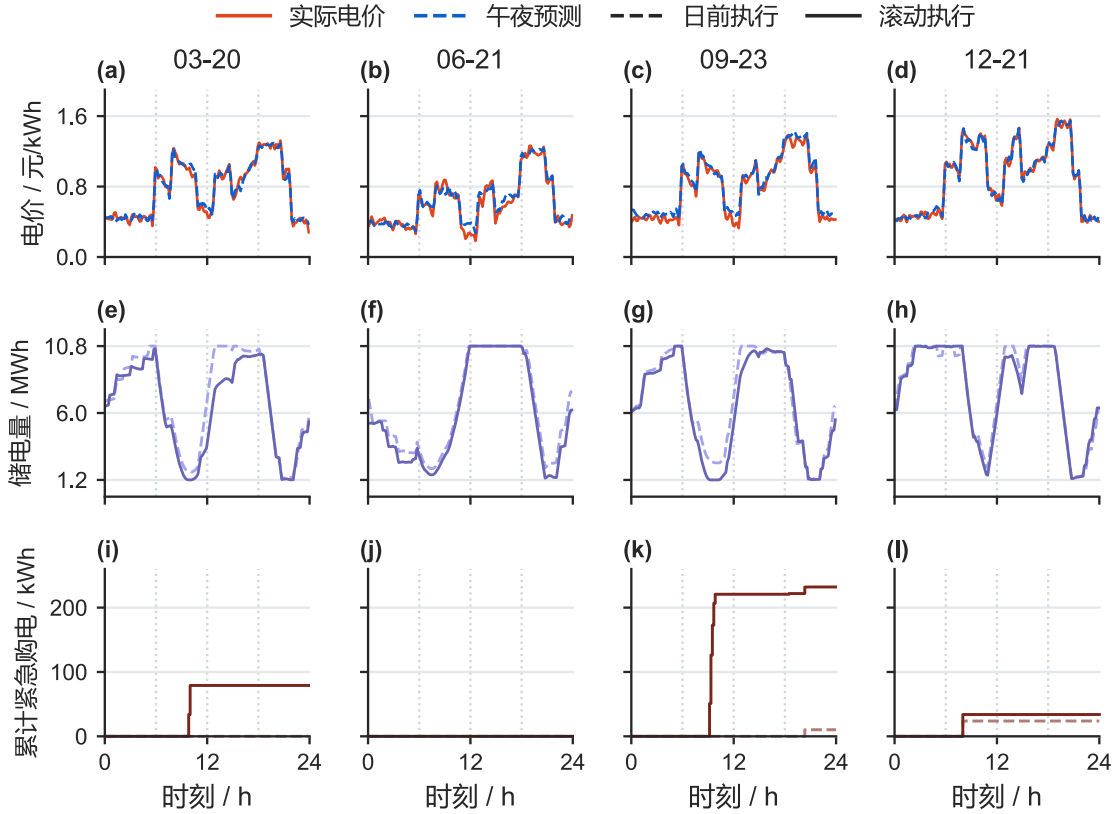


图 13 波动电价下日前与滚动策略的运行过程

图 13 第一行展示实际价格与午夜预测，第二行展示日前与滚动策略的储电量，第三行展示当日累计紧急购电量。每行共用纵轴尺度，竖虚线为 6、12、18 点。6 月 21 日两策略均未发生实质紧急购电；9 月 23 日滚动策略补购较多，说明单日效果不必与全年费用排序一致。

9.3 实际费用与调度效果

两类策略均在 2025 年 2 月 1 日至 12 月 31 日逐时执行，并保持实际储电量跨日连续。表 10 按保留合同、取消、新增及紧急补购四项归集实际费用；日前策略不能主动调单，后两类合同调整项为零。

表 10 波动电价下两类策略的实际费用构成

费用项目	日前策略/元	滚动策略/元
保留合同购电费	13,663,460.94	12,764,260.61
取消合同费用	0.00	234,774.15
新增合同购电费	0.00	765,937.23
紧急购电费	1,102,031.74	445,909.02
总费用	14,765,492.68	14,210,881.01

滚动策略总费用较日前低 554,611.67 元，降幅为 3.76%。其中紧急购电费减少 656,122.72 元，普通合同及调整相关费用合计增加 101,511.05 元：日内调整付出额外合同成本，同时降低了高价补购支出。该差额是两套完整策略的运行结果，包含预报、预测输入和调单权限的共同作用。两者期末储电量分别为 5,939.92 和 5,952.82 kWh，按题设账单不计期末残值。

图 14 进一步检验库存价值控制的贡献。在各自相同价格输入、计划生成规则和初末库存的条件下，相比即时平衡执行，日前节省 115,042.37 元，滚动节省 5,444.97 元。后者已能借助调单补足后续缺口，因此进一步优化库存配置的增益较小。

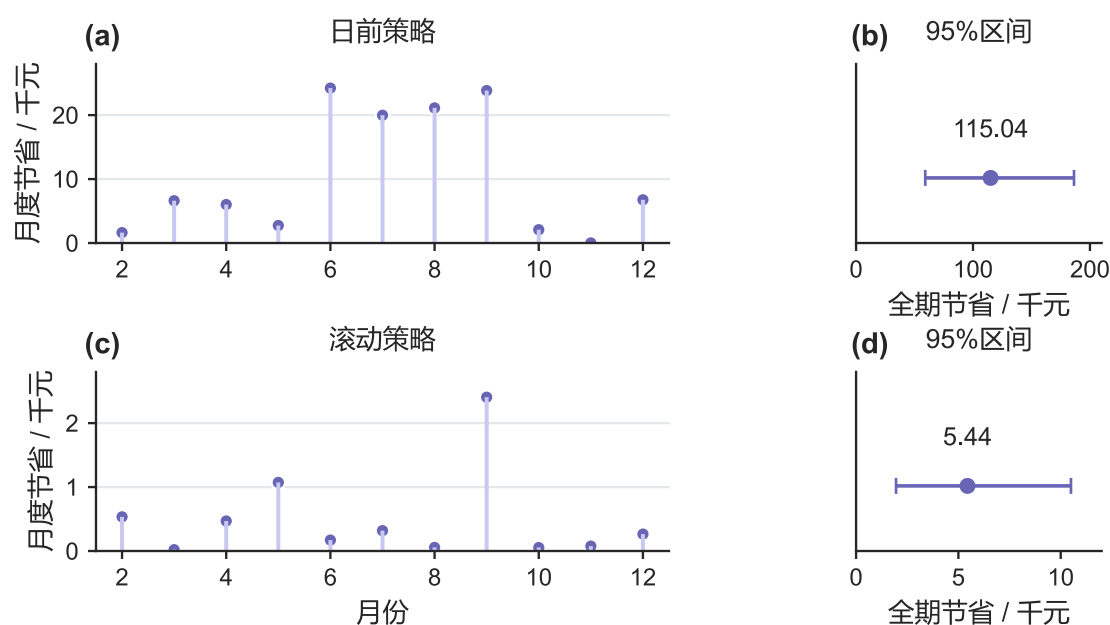


图 14 库存价值控制在波动电价下的收益

图 14 上、下两行分别为日前与滚动策略；左列为月度节省，右列为全期节省及 95% 配对区块重采样区间，均以千元为单位，各行采用独立且明确标注的线性刻度。采用 2,000 次、14 日区块，日前区间为[59,151.20, 186,368.40]元，滚动为[1,953.22, 10,500.59]元。区间反映同年固定轨迹的条件波动，不校正模型开发与选择偏差。

表 11 汇总四问的最终结果，指定日期的购电与储能表保留于附录 A。第二问到第三问扩展合同权限，第四问在这两条分支中引入波动价格，均使用相同物理约束与库存决策逻辑。

表 11 四问最终策略与购电费用

问题	新增信息或权限	最终策略	购电费用/元
一	供需与电价已知	确定性线性规划	35,126.95
二	供需预测误差	风险校准与固定合同库存价值	13,991,392.60
三	日内预报、合同调整	滚动优化与可调单库存价值	13,479,283.32
四：日前	未来电价未知	预测价格下的日前策略	14,765,492.68
四：滚动	未来电价未知	预测价格下的滚动策略	14,210,881.01

第一问为单日费用，其余为 334 天费用；各分支分别评价，不合并为同一系统的总费用。

十 模型的评价与推广

10.1 模型的优点

- （1）四问共享能量平衡与储能递推，将供需误差、预报更新及波动电价依次接入，模型扩展关系清晰，便于分析信息与合同权限的作用。
- （2）计划层以风险余量控制欠购，执行层依据后续缺口成本配置库存；保留、取消、新增与紧急补购分别结算，能够解释节费来源。
- （3）自由弃电条件下的等价线性化降低求解规模，分段线性库存价值保留连续状态，适合十分钟决策与全年滚动计算。

10.2 模型的不足

- （1）供需与电价依赖有限历史估计，残差范围不构成价格覆盖率保证；同年度开发与评价可能高估泛化效果，需要跨年度数据检验。
- （2）库存价值采用历史路径均值，滚动分支以 1.5 倍价格近似后续调单机会，未联合优化所有未来合同与库存决策，因而不保证全局最优。
- （3）模型未计入电池老化、区间内快速功率变化及网络约束。效率和合同清算口径采用工作假设，实际应用需按设备参数与交易规则校准。

10.3 模型的推广

- （1）面向园区、商业建筑或充电站，可替换负载、光伏及储能参数，在相同状态递推中加入可控充电负荷与需求响应。
- （2）可构造负载、光伏和价格的联合误差场景，以场景平均费用或风险度量扩展计划层，再用多年度数据检验风险与经济性的权衡。
- （3）可在目标中加入电池吞吐量、循环寿命、碳成本及峰值需量费用，在经济性之外评价设备寿命与减排效果。

十一 参考文献

- [1] 全国大学生数学建模竞赛组委会. 2026 年高教社杯全国大学生数学建模竞赛 C 题 微网与外部电网电力调控策略及附件.
- [2] SciPy Developers. `scipy.optimize.linprog` and `scipy.optimize.milp`.
<https://docs.scipy.org/doc/scipy/reference/optimize.html>. 访问日期 2026-09-11.
- [3] scikit-learn Developers. Ridge. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html. 访问日期 2026-09-11.
- [4] Morstyn T, Hredzak B, Aguilera R P, Agelidis V G. Model Predictive Control for Distributed Microgrid Battery Energy Storage Systems. *IEEE Transactions on Control Systems Technology*, 2018, 26(3):1107–1114. DOI:10.1109/TCST.2017.2699159.
- [5] Boyd S, Vandenberghe L. *Convex Optimization*. Cambridge University Press, 2004.
<https://web.stanford.edu/~boyd/cvxbook/>.

附录 A 四问指定日期完整结果

电量单位为 kWh，费用单位为元。普通购电与紧急购电分列。滚动问题的指定时段和全天购电量指最终有效普通计划；午夜计划及每次更新完整保存在 Excel 中。所有表由实际轨迹自动生成。

A.1 问题 1

对应结果见表 12。

表 12 问题 1（单日）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	480.41	14:00-14:10	0.00
16:00-16:10	445.43	18:00-18:10	531.89	20:00-20:10	0.00
全天购电量 59,482.70 kWh			全天购电费 35,126.95 元		

对应结果见表 13。

表 13 问题 1（单日）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,500.00	0.00	04:00-08:00	833.33	6,365.84
08:00-12:00	4,787.96	1,703.00	12:00-16:00	5,286.04	91.10
16:00-20:00	0.00	5,780.13	20:00-24:00	5,333.33	2,859.87
00:00 储电量	6,000.00		24:00 储电量	6,000.00	

A.2 问题 2

对应结果见表 14。

表 14 问题 2（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	720.76	20:00-20:10	0.00
全天购电量 68,036.47 kWh			全天购电费 41,302.44 元		

对应结果见表 15。

表 15 问题 2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,071.86	209.79	04:00-08:00	1,038.99	5,515.86
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	266.98	5,654.71	20:00-24:00	5,687.88	2,954.47
00:00 储电	6,617.05		24:00 储电	6,382.68	

对应结果见表 16。

表 16 问题 2（2025-03-20）紧急购电

时段	电量
17:20-17:30	5.19

对应结果见表 17。

表 17 问题 2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,315.40 kWh			全天购电费 21,120.39 元		

对应结果见表 18。

表 18 问题 2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	526.76	524.94	04:00-08:00	413.77	4,402.74
08:00-12:00	9,426.35	0.00	12:00-16:00	0.00	0.00
16:00-20:00	151.96	5,493.83	20:00-24:00	6,132.83	2,497.81
00:00 储电	6,945.01		24:00 储电	7,576.71	

对应结果见表 19。

表 19 问题 2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 20。

表 20 问题 2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	788.73	20:00-20:10	0.00
全天购电量 71,561.09 kWh			全天购电费 44,415.98 元		

对应结果见表 21。

表 21 问题 2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,252.67	0.00	04:00-08:00	218.14	5,830.96
08:00-12:00	5,558.85	1,896.62	12:00-16:00	4,033.40	575.17
16:00-20:00	231.66	5,642.21	20:00-24:00	6,050.86	2,841.96
00:00 储电	6,072.60		24:00 储电	6,631.50	

对应结果见表 22。

表 22 问题 2（2025-09-23）紧急购电

时段	电量
20:20-20:30	0.33

对应结果见表 23。

表 23 问题 2（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,145.51 kWh		全天购电费 62,434.14 元			

对应结果见表 24。

表 24 问题 2（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,629.25	143.74	04:00-08:00	1,398.62	1,538.91
08:00-12:00	5,704.82	7,168.67	12:00-16:00	6,940.07	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,695.76	2,842.53
00:00 储电	6,318.60		24:00 储电	6,443.34	

对应结果见表 25。

表 25 问题 2（2025-12-21）紧急购电

时段	电量
07:50-08:00	23.78

A.3 问题 3

对应结果见表 26。

表 26 问题 3（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	520.67	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	0.00
全天购电量 66,401.94 kWh		全天购电费 42,938.13 元			

对应结果见表 27。

表 27 问题 3（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.82	348.06	04:00-08:00	1,010.91	5,813.27
08:00-12:00	3,254.96	2,698.33	12:00-16:00	6,483.14	254.72
16:00-20:00	646.16	5,206.99	20:00-24:00	5,530.12	3,018.64
00:00 储电	6,309.41		24:00 储电	6,100.33	

对应结果见表 28。

表 28 问题 3（2025-03-20）紧急购电

时段	电量
09:40-10:00	79.05
21:40-21:50	4.66

对应结果见表 29。

表 29 问题 3（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	120.43	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,874.69 kWh			全天购电费 19,723.09 元		

对应结果见表 30。

表 30 问题 3（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	271.67	04:00-08:00	449.87	3,308.89
08:00-12:00	9,888.78	0.00	12:00-16:00	0.00	0.00
16:00-20:00	41.37	6,074.66	20:00-24:00	5,603.72	2,654.17
00:00 储电	5,225.44		24:00 储电	6,181.87	

对应结果见表 31。

表 31 问题 3（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 32。

表 32 问题 3（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	522.47	18:00-18:10	784.63	20:00-20:10	0.00
全天购电量 67,918.94 kWh			全天购电费 44,580.19 元		

对应结果见表 33。

表 33 问题 3（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,885.98	16.12	04:00-08:00	353.76	6,967.82
08:00-12:00	4,885.84	1,675.86	12:00-16:00	6,129.08	508.84
16:00-20:00	118.44	5,841.36	20:00-24:00	5,814.52	2,695.27
00:00 储电	6,102.47		24:00 储电	6,398.80	

对应结果见表 34。

表 34 问题 3（2025-09-23）紧急购电

时段	电量
09:00-09:50	220.76
20:20-20:30	6.37

对应结果见表 35。

表 35 问题 3（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,721.86 kWh		全天购电费 62,219.35 元			

对应结果见表 36。

表 36 问题 3（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,657.30	179.36	04:00-08:00	1,565.97	1,572.10
08:00-12:00	5,310.40	7,513.79	12:00-16:00	7,773.62	2,250.94
16:00-20:00	57.98	5,766.67	20:00-24:00	5,676.93	2,842.53
00:00 储电	6,219.20		24:00 储电	6,395.64	

对应结果见表 37。

表 37 问题 3（2025-12-21）紧急购电

时段	电量
07:50-08:00	33.89

A.4-2 问题 4-2

对应结果见表 38。

表 38 问题 4-2（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	574.14	14:00-14:10	0.00
16:00-16:10	488.93	18:00-18:10	310.56	20:00-20:10	756.24
全天购电量 67,791.08 kWh		全天购电费 42,355.14 元			

对应结果见表 39。

表 39 问题 4-2（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,655.77	152.63	04:00-08:00	1,195.22	5,572.84
08:00-12:00	6,186.28	2,777.38	12:00-16:00	4,162.63	444.57
16:00-20:00	214.58	7,108.02	20:00-24:00	5,717.23	1,458.76
00:00 储电	6,850.72		24:00 储电	6,409.05	

对应结果见表 40。

表 40 问题 4-2（2025-03-20）紧急购电

时段	电量
无	0.00

对应结果见表 41。

表 41 问题 4-2（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	210.92	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 36,422.89 kWh			全天购电费 18,518.97 元		

对应结果见表 42。

表 42 问题 4-2（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	438.52	3,694.59	04:00-08:00	1,239.76	1,877.96
08:00-12:00	9,421.97	0.00	12:00-16:00	0.00	0.00
16:00-20:00	163.47	5,505.34	20:00-24:00	6,196.26	2,497.81
00:00 储电	7,001.50		24:00 储电	7,631.37	

对应结果见表 43。

表 43 问题 4-2（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 44。

表 44 问题 4-2（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	491.40	14:00-14:10	0.00
16:00-16:10	549.29	18:00-18:10	416.70	20:00-20:10	0.00
全天购电量 71,436.45 kWh			全天购电费 46,451.24 元		

对应结果见表 45。

表 45 问题 4-2（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,585.50	0.00	04:00-08:00	760.67	5,830.96
08:00-12:00	5,687.53	1,896.62	12:00-16:00	3,858.02	528.46
16:00-20:00	253.12	6,023.40	20:00-24:00	6,035.33	2,472.31
00:00 储电	6,184.77		24:00 储电	6,633.87	

对应结果见表 46。

表 46 问题 4-2（2025-09-23）紧急购电

时段	电量
20:10-20:20	10.23

对应结果见表 47。

表 47 问题 4-2（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	967.61	14:00-14:10	0.00
16:00-16:10	812.86	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 97,314.24 kWh			全天购电费 73,263.64 元		

对应结果见表 48。

表 48 问题 4-2（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,056.07	143.74	04:00-08:00	1,743.90	2,155.89
08:00-12:00	5,755.95	7,219.80	12:00-16:00	6,952.06	2,243.11
16:00-20:00	24.16	5,708.86	20:00-24:00	5,755.02	2,845.49
00:00 储电	6,309.24		24:00 储电	6,493.37	

对应结果见表 49。

表 49 问题 4-2（2025-12-21）紧急购电

时段	电量
07:50-08:00	23.78

A.4-3 问题 4-3

对应结果见表 50。

表 50 问题 4-3（2025-03-20）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	520.67	14:00-14:10	0.00
16:00-16:10	638.18	18:00-18:10	714.94	20:00-20:10	741.44
全天购电量 66,091.31 kWh			全天购电费 43,656.33 元		

对应结果见表 51。

表 51 问题 4-3（2025-03-20）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	3,255.66	249.34	04:00-08:00	1,840.75	5,937.64
08:00-12:00	3,254.96	2,698.33	12:00-16:00	6,863.61	254.72
16:00-20:00	291.94	6,696.45	20:00-24:00	5,561.08	1,554.74
00:00 储电	6,485.79		24:00 储电	6,123.41	

对应结果见表 52。

表 52 问题 4-3（2025-03-20）紧急购电

时段	电量
09:40-10:00	79.05

对应结果见表 53。

表 53 问题 4-3（2025-06-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	0.00	14:00-14:10	0.00
16:00-16:10	120.43	18:00-18:10	0.00	20:00-20:10	0.00
全天购电量 34,998.69 kWh			全天购电费 17,375.08 元		

对应结果见表 54。

表 54 问题 4-3（2025-06-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	275.76	2,705.06	04:00-08:00	1,497.17	1,779.01
08:00-12:00	9,921.65	0.00	12:00-16:00	0.00	0.00
16:00-20:00	41.37	6,074.66	20:00-24:00	5,613.02	2,623.77
00:00 储电	5,257.18		24:00 储电	6,224.03	

对应结果见表 55。

表 55 问题 4-3（2025-06-21）紧急购电

时段	电量
无	0.00

对应结果见表 56。

表 56 问题 4-3（2025-09-23）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	347.56	14:00-14:10	0.00
16:00-16:10	522.47	18:00-18:10	628.48	20:00-20:10	0.00
全天购电量 67,886.23 kWh			全天购电费 46,606.18 元		

对应结果见表 57。

表 57 问题 4-3（2025-09-23）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	4,249.37	22.22	04:00-08:00	984.97	6,967.82
08:00-12:00	4,885.84	1,675.86	12:00-16:00	6,081.90	461.66
16:00-20:00	110.65	5,980.56	20:00-24:00	5,668.05	2,563.12
00:00 储电	6,114.10		24:00 储电	6,262.09	

对应结果见表 58。

表 58 问题 4-3（2025-09-23）紧急购电

时段	电量
09:00-09:50	220.76
18:20-18:30	1.08
20:10-20:20	10.23

对应结果见表 59。

表 59 问题 4-3（2025-12-21）指定时段购电及全天费用

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.00	12:00-12:10	924.13	14:00-14:10	0.00
16:00-16:10	821.56	18:00-18:10	739.98	20:00-20:10	0.00
全天购电量 95,641.71 kWh			全天购电费 72,890.05 元		

对应结果见表 60。

表 60 问题 4-3（2025-12-21）储能运行

时间段	充电量	放电量	时间段	充电量	放电量
00:00-04:00	5,169.62	178.51	04:00-08:00	232.98	904.57
08:00-12:00	5,347.35	7,538.30	12:00-16:00	8,080.06	2,504.57
16:00-20:00	57.98	5,766.67	20:00-24:00	5,708.45	2,845.49
00:00 储电	6,215.16		24:00 储电	6,420.72	

对应结果见表 61。

表 61 问题 4-3（2025-12-21）紧急购电

时段	电量
07:50-08:00	33.89

附录 B 支撑文件与计算程序

五份结果工作簿分别为 result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx 和 result4-3.xlsx，均保持原题模板。完整运行数据、代码和图表源文件随支撑材料提供；程序按“数据读取—历史预测—风险校准—计划求解—实时执行—费用结算”的顺序组织。

下面给出价格预测、线性规划及库存价值计算的核心程序。所有实际费用以交易时刻价格结算，原始附件只读。

src/q12.py

```
"""Causal forecasts, MILP day-ahead dispatch and independent physical audits.

Run from the repository: python -m src.q12 --data-root <C 题 directory> --stage full
All power observations are interpreted as interval means at right-end labels.
"""

from __future__ import annotations

import argparse
from dataclasses import asdict, dataclass
import hashlib
import json
import os
from pathlib import Path
import platform
import subprocess
import time

os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")
os.environ.setdefault("OMP_NUM_THREADS", "1")
import numpy as np
import pandas as pd
import scipy
from scipy.optimize import Bounds, LinearConstraint, milp
from scipy.sparse import lil_matrix

@dataclass(frozen=True)
class Battery:
    minimum: float = 1200.0
    maximum: float = 10800.0
    initial: float = 6000.0
    max_power: float = 5000.0
    eta_c: float = 0.9
    eta_d: float = 0.9
    dt: float = 1 / 6

    @property
    def limit(self):
        return self.max_power * self.dt

B = Battery()
N = 144
TOL = 1e-5

def dump_json(path, obj):
    Path(path).write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False), encoding="utf-8")

def interval_label(i):
    def clock(m):
        return f"{m // 60:02d}:{m % 60:02d}"
    return f"{clock(i * 10)}-{clock((i + 1) * 10)}"
```

```

def time_minutes(value):
    if hasattr(value, "hour"):
        return value.hour*60 + value.minute
    text = str(value).strip()
    next_day = "+1" in text
    hour, minute, *_ = text.replace("+1", "").split(":")
    return int(hour)*60 + int(minute) + 1440*int(next_day)

def read_inputs(root):
    root = Path(root)
    day = pd.read_excel(root / "附件/附件 1.xlsx", sheet_name=0, header=0)
    load = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="小区负载", header=0)
    pv = pd.read_excel(root / "附件/附件 2.xlsx", sheet_name="光伏发电实际功率", header=0)
    dates = pd.to_datetime(load.iloc[:, 0])
    if day.shape != (144, 4) or load.shape != (365, 145) or pv.shape != (365, 145):
        raise ValueError("Unexpected input dimensions")
    if not dates.equals(pd.to_datetime(pv.iloc[:, 0])):
        raise ValueError("Load/PV dates differ")
    if list(dates) != list(pd.date_range("2025-01-01", "2025-12-31")):
        raise ValueError("Dates must cover the complete ordered year")
    expected_minutes = list(range(10, 1441, 10))
    if any([time_minutes(v) for v in labels] != expected_minutes
           for labels in (load.columns[1:], pv.columns[1:], day.iloc[:, 0])):
        raise ValueError("Input time labels do not match")
    data = {
        "dates": pd.DatetimeIndex(dates), "price": day.iloc[:, 1].to_numpy(float),
        "q1_load": day.iloc[:, 2].to_numpy(float) * B.dt,
        "q1_pv": day.iloc[:, 3].to_numpy(float) * B.dt,
        "load": load.iloc[:, 1:].to_numpy(float) * B.dt,
        "pv": pv.iloc[:, 1:].to_numpy(float) * B.dt,
    }
    audit = {"shapes": {"attachment1": list(day.shape), "load": list(load.shape), "pv": list(pv.shape)},
            "date_start": str(dates.iloc[0].date()), "date_end": str(dates.iloc[-1].date()),
            "duplicates": int(dates.duplicated().sum()), "input_files": [], "variables": {}}
    for key in ["price", "q1_load", "q1_pv", "load", "pv"]:
        a = data[key]
        if not np.isfinite(a).all() or (a < 0).any():
            raise ValueError(f"Nonfinite or negative data in {key}; investigate, do not silently impute")
        audit["variables"][key] = {"count": int(a.size), "missing": 0, "min": float(a.min()),
                                   "max": float(a.max()), "zero_count": int((a == 0).sum())}
    if (data["price"] <= 0).any():
        raise ValueError("This implementation assumes strictly positive fixed prices")
    for relative in ["C 题.pdf", "附件/附件 1.xlsx", "附件/附件 2.xlsx"]:
        raw = (root / relative).read_bytes()
        audit["input_files"].append({"path": relative, "bytes": len(raw), "sha256":
                                     hashlib.sha256(raw).hexdigest()})
    audit["data_treatment"] = "No missing/negative values found; no observations deleted or imputed. Unusual
    peaks retained."
    return data, audit

def optimize_day(load, pv, price, initial, terminal=6000.0, battery=B, integer=True):
    """Nominal minimum-cost schedule, g/c/d/w/S/z in kWh; terminal is a lower bound.

    For Q1 initial=terminal and terminal_equal=True is obtained by the explicit
    upper bound below when requested by a (value, 'equal') tuple.
    """
    n = len(price)
    eq_terminal = isinstance(terminal, tuple)
    target = terminal[0] if eq_terminal else terminal
    size = 6 * n + 1
    gi, ci, di, wi, si, zi = 0, n, 2*n, 3*n, 4*n, 5*n+1
    objective = np.zeros(size)
    objective[:n] = price
    lb, ub = np.zeros(size), np.full(size, np.inf)
    ub[ci:wi] = battery.limit
    lb[si:zi], ub[si:zi] = battery.minimum, battery.maximum
    lb[si] = ub[si] = initial
    lb[si+n] = target
    if eq_terminal:
        ub[si+n] = target
    ub[zi:] = 1
    integrality = np.zeros(size)
    integrality[zi:] = int(integer)
    a = lil_matrix((4*n, size))

```

```

lower, upper = np.zeros(4*n), np.zeros(4*n)
for t in range(n):
    a[t, gi+t], a[t, ci+t], a[t, di+t], a[t, wi+t] = 1, -1, 1, -1
    lower[t] = upper[t] = load[t] - pv[t]
    a[n+t, si+t+1], a[n+t, si+t] = 1, -1
    a[n+t, ci+t], a[n+t, di+t] = -battery.eta_c, 1 / battery.eta_d
    a[2*n+t, ci+t], a[2*n+t, zi+t] = 1, -battery.limit
    lower[2*n+t], upper[2*n+t] = -np.inf, 0
    a[3*n+t, di+t], a[3*n+t, zi+t] = 1, battery.limit
    lower[3*n+t], upper[3*n+t] = -np.inf, battery.limit
start = time.perf_counter()
result = milp(objective, integrality=integrality, bounds=Bounds(lb, ub),
              constraints=LinearConstraint(a.tocsc(), lower, upper),
              options={"time_limit": 30.0, "mip_rel_gap": 1e-7})
elapsed = time.perf_counter() - start
if not result.success or result.x is None:
    raise RuntimeError(f"MILP failed: status={result.status}, {result.message}")
x = result.x
sol = {"plan_kwh": x[:n], "charge_kwh": x[ci:di], "discharge_kwh": x[di:wi],
      "spill_kwh": x[wi:si], "soc_start_kwh": x[si:si+n], "soc_end_kwh": x[si+1:zi],
      "emergency_kwh": np.zeros(n), "load_kwh": np.asarray(load), "pv_kwh": np.asarray(pv)}
frame = pd.DataFrame(sol)
frame["price"] = price
meta = {"status": int(result.status), "cost": float(np.dot(price, x[:n])),
      "seconds": elapsed, "mip_gap": float(getattr(result, "mip_gap", 0) or 0),
      "dual_bound": float(getattr(result, "mip_dual_bound", result.fun) or result.fun)}
if integer:
    audit_trace(frame, battery=battery, initial=initial)
return frame, meta

def execute_day(plan, load, pv, initial, price, battery=B):
    """Causal real-time balancing; does not read future observations.

    Assumes instantaneous regulation within a ten-minute constant-power interval.
    Surplus charges first; deficit exhausts feasible discharge before emergency.
    """
    s = float(initial)
    records = []
    for t, (g, l, v) in enumerate(zip(plan, load, pv)):
        surplus = float(g + v - l)
        c = min(max(surplus, 0), battery.limit, max(0, (battery.maximum-s)/battery.eta_c))
        d = min(max(-surplus, 0), battery.limit, max(0, (s-battery.minimum)*battery.eta_d))
        e = max(0, -surplus-d)
        w = max(0, surplus-c)
        end = s + battery.eta_c*c - d/battery.eta_d
        records.append((g, l, v, c, d, e, w, s, end, price[t]))
        s = end
    frame = pd.DataFrame(records, columns=["plan_kwh", "load_kwh", "pv_kwh", "charge_kwh", "discharge_kwh",
      "emergency_kwh", "spill_kwh", "soc_start_kwh", "soc_end_kwh",
"price"], dtype=float)
    audit_trace(frame, battery=battery, initial=initial)
    return frame

def audit_trace(frame, battery=B, initial=None):
    """Independent vectorized equations, also applicable to saved full-year traces."""
    f = frame
    balance = f.plan_kwh + f.pv_kwh + f.discharge_kwh + f.emergency_kwh - f.load_kwh - f.charge_kwh -
f.spill_kwh
    evolution = f.soc_end_kwh - f.soc_start_kwh - battery.eta_c*f.charge_kwh + f.discharge_kwh/battery.eta_d
    continuity = f.soc_start_kwh.to_numpy()[1:] - f.soc_end_kwh.to_numpy()[:-1]
    metrics = {"max_balance_residual_kwh": float(np.max(np.abs(balance))),
      "max_soc_residual_kwh": float(np.max(np.abs(evolution))),
      "max_soc_discontinuity_kwh": float(np.max(np.abs(continuity))) if len(f)>1 else 0,
      "simultaneous_intervals": int(((f.charge_kwh > TOL) & (f.discharge_kwh > TOL)).sum()),
      "min_soc_kwh": float(min(f.soc_start_kwh.min(), f.soc_end_kwh.min())),
      "max_soc_kwh": float(max(f.soc_start_kwh.max(), f.soc_end_kwh.max()))}
    checks = [np.isfinite(f.select_dtypes(include="number").to_numpy()).all(),
      (f[["plan_kwh", "charge_kwh", "discharge_kwh", "emergency_kwh", "spill_kwh"]].to_numpy() >= -
TOL).all(),
      metrics["max_balance_residual_kwh"] < TOL, metrics["max_soc_residual_kwh"] < TOL,
      metrics["max_soc_discontinuity_kwh"] < TOL, metrics["simultaneous_intervals"] == 0,
      metrics["min_soc_kwh"] >= battery.minimum-TOL, metrics["max_soc_kwh"] <= battery.maximum+TOL,
      f.charge_kwh.max() <= battery.limit+TOL, f.discharge_kwh.max() <= battery.limit+TOL]
    if initial is not None:
        checks.append(abs(f.soc_start_kwh.iloc[0]-initial) < TOL)
    if not all(checks):

```

```

        raise AssertionError(f"Physical audit failed: {metrics}")
    return {**metrics, "pass": True}

def features(history, day, dates):
    """Features for all 144 targets on a day, using completed days only."""
    if day < 7:
        raise ValueError("Seven completed days required")
    past = history[day-7:day]
    h = (np.arange(N)+0.5)/N
    dow = dates[day].dayofweek
    cols = [past[-1], past[-2], past[0], past[-3:].mean(0), past.mean(0), past.std(0),
            np.full(N, past[-1].mean()), np.full(N, past[-3:].mean())]
    for k in (1, 2, 3):
        cols.extend([np.sin(2*np.pi*k*h), np.cos(2*np.pi*k*h)])
    cols.extend([np.full(N, float(dow == k)) for k in range(7)])
    return np.column_stack(cols)

def ridge_fit(x, y, alpha):
    """Centered ridge via linear solve; intercept unpenalized, no sklearn required."""
    mean, scale = x.mean(0), x.std(0)
    scale[scale < 1e-10] = 1
    z = (x-mean)/scale
    intercept = float(y.mean())
    coef = np.linalg.solve(z.T@z + alpha*np.eye(z.shape[1]), z.T@(y-intercept))
    return mean, scale, coef, intercept

def forecasts(data, method, alphas=(10.0, 10.0), stop=365):
    pred, fit_dates = {}, []
    for k, alpha in zip(("load", "pv"), alphas):
        values = data[k]
        out = np.zeros((stop, N))
        model, fitted_on = None, -1
        all_x = {d: features(values, d, data["dates"]) for d in range(7, stop)}
        for d in range(stop):
            if d == 0:
                out[d] = data[f"q1_{k}"]
            elif d < 7:
                out[d] = values[max(0,d-3):d].mean(0)
            elif method == "seasonal" or d < 14:
                out[d] = 0.5*values[d-1] + 0.5*values[d-7]
            else:
                if model is None or d-fitted_on >= 7:
                    training_days = range(max(7,d-60), d)
                    x = np.concatenate([all_x[j] for j in training_days])
                    y = values[max(7,d-60):d].ravel()
                    model = ridge_fit(x, y, alpha)
                    fitted_on = d
                fit_dates.append({"variable": k, "decision_date": str(data["dates"][d].date()),
                                   "fit_until": str(data["dates"][d-1].date()), "alpha": alpha,
                                   "training_rows": len(y)})
                mean, scale, coef, intercept = model
                out[d] = ((all_x[d]-mean)/scale)@coef+intercept
            out[d] = np.maximum(out[d], 0)
            if k == "pv" and d:
                out[d, values[max(0,d-7):d].max(0) == 0] = 0
        pred[k] = out
    return pred, fit_dates

def reserve(data, forecast, day, quantile):
    if quantile == 0 or day < 7:
        return np.zeros(N)
    start = max(1, day-28)
    errors = (data["load"][start:day]-data["pv"][start:day]
              -forecast["load"][start:day]+forecast["pv"][start:day])
    return np.array([max(0.0, float(np.quantile(errors[:,max(0,t-3):min(N,t+4)], quantile))) for t in
                     range(N)])

def summarize_day(f, date, name, seconds, gap=0.0):
    normal = float(np.dot(f.plan_kwh, f.price))
    emergency = float(5*np.dot(f.emergency_kwh, f.price))
    return {"date": str(date.date()), "strategy": name, "normal_cost_yuan": normal,

```

```

        "emergency_cost_yuan": emergency, "total_cost_yuan": normal+emergency,
        "plan_kwh": float(f.plan_kwh.sum()), "emergency_kwh": float(f.emergency_kwh.sum()),
        "emergency_intervals": int((f.emergency_kwh>TOL).sum()), "spill_kwh": float(f.spill_kwh.sum()),
        "charge_kwh": float(f.charge_kwh.sum()), "discharge_kwh": float(f.discharge_kwh.sum()),
        "soc_start_kwh": float(f.soc_start_kwh.iloc[0]), "soc_end_kwh": float(f.soc_end_kwh.iloc[-1]),
        "solve_seconds": seconds, "mip_gap": gap}

def backtest(data, forecast, start, stop, initial, quantile, name, retain=True):
    s, daily, frames = initial, [], []
    for d in range(start,stop):
        buffer = reserve(data, forecast, d, quantile)
        nominal, meta = optimize_day(forecast["load"][d]+buffer, forecast["pv"][d], data["price"], s)
        f = execute_day(nominal.plan_kwh.to_numpy(), data["load"][d], data["pv"][d], s, data["price"])
        daily.append(summarize_day(f, data["dates"][d], name, meta["seconds"], meta["mip_gap"]))
        if retain:
            f.insert(0,"interval_start", pd.date_range(data["dates"][d],periods=N,freq="10min"))
            f.insert(1,"date", str(data["dates"][d].date()))
            f.insert(2,"slot", np.arange(N))
            f["forecast_load_kwh"], f["forecast_pv_kwh"], f["reserve_kwh"] = forecast["load"][d],
forecast["pv"][d], buffer
            frames.append(f)
            s = float(f.soc_end_kwh.iloc[-1])
            if retain and (d == start or data["dates"][d].day == 1):
                print(f"{name}: {data['dates'][d].date()}, cumulative cost={sum(x['total_cost_yuan'] for x in
daily):.2f}", flush=True)
            combined = pd.concat(frames,ignore_index=True) if frames else None
            if combined is not None:
                audit_trace(combined, initial=initial)
            return pd.DataFrame(daily), combined, s

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--data-root", type=Path, required=True)
    parser.add_argument("--out", type=Path, default=Path("artifacts/q12"))
    parser.add_argument("--stage", choices=["q1","smoke","full"],default="full")
    args = parser.parse_args()
    args.out.mkdir(parents=True,exist_ok=True)
    start = time.perf_counter()
    data, input_audit = read_inputs(args.data_root)
    dump_json(args.out/"input_audit.json", input_audit)
    q1, q1_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial,"equal"))
    lp, lp_meta = optimize_day(data["q1_load"], data["q1_pv"], data["price"], B.initial, (B.initial,"equal"),
integer=False)
    q1.insert(0,"slot",np.arange(N));q1.insert(1,"interval",[interval_label(i) for i in range(N)])
    q1.to_csv(args.out/"q1_intervals.csv",index=False)
    no_storage_cost = float(np.dot(np.maximum(data["q1_load"]-data["q1_pv"],0),data["price"]))
    q1_meta.update({"no_storage_cost":no_storage_cost,"saving_percent":100*(no_storage_cost-
q1_meta["cost"])/no_storage_cost,
                    "lp_cost_lower_bound":lp_meta["cost"], "lp_gap_yuan": q1_meta["cost"]-lp_meta["cost"],
                    "audit":audit_trace(q1,initial=B.initial), "plan_kwh":float(q1.plan_kwh.sum())})
    dump_json(args.out/"q1_summary.json",q1_meta)
    print("Q1",json.dumps(q1_meta,ensure_ascii=False),flush=True)
    if args.stage == "q1":
        return
    seasonal, _ = forecasts(data,"seasonal")
    tuning = []
    for alpha in (1.0,10.0,100.0):
        pred, _ = forecasts(data,"ridge",(alpha,alpha),stop=31)
        row = {"alpha":alpha}
        for key in ("load","pv"):
            row[key+"_mae_kw"] = float(np.abs(pred[key][21:31]-data[key][21:31]).mean())/B.dt)
        tuning.append(row)
    alpha_load = min(tuning,key=lambda r:r["load_mae_kw"])[["alpha"]]
    alpha_pv = min(tuning,key=lambda r:r["pv_mae_kw"])[["alpha"]]
    ridge, fit_log = forecasts(data,"ridge",(alpha_load,alpha_pv))
    dump_json(args.out/"forecast_fit_log.json",fit_log)
    pd.DataFrame(tuning).to_csv(args.out/"ridge_validation.csv",index=False)
    _, _, validation_soc = backtest(data,seasonal,0,21,B.initial,0,"warmup",retain=False)
    warmup, _, evaluation_soc = backtest(data,seasonal,21,31,validation_soc,0,"warmup",retain=False)
    warmup.to_csv(args.out/"warmup_last10days.csv",index=False)
    validation = []
    for method, pred in (("seasonal",seasonal),("ridge",ridge)):
        for q in (0.0,0.7,0.8,0.9):
            daily, _, _ = backtest(data,pred,21,31,validation_soc,q,method,retain=False)
            validation.append({"forecast":method,"quantile":q,"cost_yuan":float(daily.total_cost_yuan.sum()),
                             "emergency_cost_yuan":float(daily.emergency_cost_yuan.sum()),
                             "end_soc_kwh":float(daily.soc_end_kwh.iloc[-1])})

```

```

selected = min(validation, key=lambda r: r["cost_yuan"])
pd.DataFrame(validation).to_csv(args.out/"policy_validation.csv", index=False)
stop = 38 if args.stage == "smoke" else 365
strategies = [("seasonal_q0", "seasonal", 0.0), ("ridge_q0", "ridge", 0.0)]
for method in ("seasonal", "ridge"):
    best = min((v for v in validation if v["forecast"] == method), key=lambda r: r["cost_yuan"])
    if best["quantile"]:
        strategies.append((f"{method}_q{best['quantile']:g}", method, best["quantile"]))
selected_name = f"{selected['forecast']}_q{selected['quantile']:g}"
all_daily, metrics = [], []
for name, method, q in strategies:
    pred = {"seasonal": seasonal, "ridge": ridge}[method]
    daily, trace, _ = backtest(data, pred, 31, stop, evaluation_soc, q, name)
    all_daily.append(daily)
    trace.to_csv(args.out/f"{name}_intervals.csv.gz", index=False, float_format="%.17g")
    audit = audit_trace(trace, initial=evaluation_soc)
    row = {"strategy": name, "forecast": method, "quantile": q, "selected_on_january": name == selected_name,
          "days": len(daily), "total_cost_yuan": float(daily.total_cost_yuan.sum()),
          "normal_cost_yuan": float(daily.normal_cost_yuan.sum()),
          "emergency_cost_yuan": float(daily.emergency_cost_yuan.sum()),
          "emergency_kwh": float(daily.emergency_kwh.sum()), "emergency_intervals": int(daily.emergency_intervals.sum()),
          "plan_kwh": float(daily.plan_kwh.sum()), "spill_kwh": float(daily.spill_kwh.sum()),
          "solve_seconds": float(daily.solve_seconds.sum()), "initial_soc_kwh": evaluation_soc,
          "final_soc_kwh": float(daily.soc_end_kwh.iloc[-1]), "audit": audit}
    for key in ("load", "pv"):
        error = (pred[key][31:stop] - data[key][31:stop]) / B.dt
        row[key + "_mae_kw"], row[key + "_rmse_kw"] = float(np.abs(error).mean()), float(np.sqrt(np.mean(error**2)))
    metrics.append(row)
daily = pd.concat(all_daily, ignore_index=True)
daily.to_csv(args.out/"q2_daily_metrics.csv", index=False)

dump_json(args.out/"q2_summary.json", {"selection": selected, "selected_strategy": selected_name, "strategies": metrics})
command = f'python -m src.q12 --data-root "{args.data_root}" --out "{args.out}" --stage {args.stage}'
manifest = {"stage": args.stage, "battery": asdict(B), "time_interpretation": "interval mean power, right-end labels",
           "alpha_load": alpha_load, "alpha_pv": alpha_pv, "validation_start": "2025-01-22", "validation_end": "2025-01-31",
           "evaluation_start": "2025-02-01", "evaluation_end": str(data["dates"][stop-1].date()),
           "validation_initial_soc": validation_soc, "evaluation_initial_soc": evaluation_soc,
           "terminal_nominal_soc_minimum": 6000, "refit_period_days": 7, "training_window_days": 60,
           "residual_window_days": 28, "pool_halfwidth_slots": 3, "seed": 2026, "deterministic": True,
           "selected_strategy": selected_name, "command": command, "total_seconds": time.perf_counter() - start,
           "python": platform.python_version(), "numpy": np.__version__, "scipy": scipy.__version__, "pandas": pd.__version__,
           "platform": platform.platform(), "git_head_before_run": subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).strip(),
           "source_sha256": hashlib.sha256(Path(__file__).read_bytes()).hexdigest(), "input_files": input_audit["input_files"]}
dump_json(args.out/"run_manifest.json", manifest)
print("COMPLETE", json.dumps({"selected": selected_name, "seconds": manifest["total_seconds"], "days": stop-31}), flush=True)

if __name__ == "__main__":
    main()

```

src/q34_data.py

```

"""Strict attachment 3/4 parsing and causal hourly forecast alignment."""
from pathlib import Path
import hashlib
import numpy as np
import pandas as pd
import openpyxl
from src.q12 import read_inputs, time_minutes, dump_json

def read_extended(root, audit_path=None):
    root = Path(root); data, base_audit = read_inputs(root)
    wb = openpyxl.load_workbook(root/'附件/附件 3.xlsx', read_only=True, data_only=True)
    rows = list(wb.active.values); wb.close()
    assert len(rows) == 1461 and len(rows[0]) == 26
    assert list(rows[0][2:]) == [f'预报{i}小时' for i in range(1, 25)]

```

```

hourly=np.empty((365,4,24)); filled=0
for day in range(365):
    date=data['dates'][day]
    for version in range(4):
        row=rows[1+day*4+version]
        if version==0:
            assert pd.Timestamp(row[0])==date
        elif row[0] in (None,''):
            filled+=1
        else:assert pd.Timestamp(row[0])==date
        assert time_minutes(row[1])==version*360
        hourly[day,version]=np.asarray(row[2:],float)
wb=openpyxl.load_workbook(root/'附件/附件 4.xlsx',read_only=True,data_only=True)
rows=list(wb.active.values);wb.close()
assert len(rows)==366 and len(rows[0])==145
assert [time_minutes(v) for v in rows[0][1:]]==list(range(10,1441,10))
assert pd.DateTimeIndex([r[0] for r in rows[1:]].equals(data['dates'])
prices=np.asarray([r[1:] for r in rows[1:]],float)
checks={}
for name,values in [('hourly_pv_kw',hourly),('actual_price',prices)]:
    assert np.isfinite(values).all(),f'{name}: missing/nonfinite'
    assert (values>=0).all(),f'{name}: negative values need investigation'
    if name=='actual_price':assert (values>0).all()
    q1,q3=np.quantile(values,[.25,.75]);iqr=q3-q1
    checks[name]={ 'shape':list(values.shape), 'count':values.size, 'missing_nonfinite':0,
        'negative':0, 'zero':int((values==0).sum()), 'min':float(values.min()), 'max':float(values.max()),
        'iqr_flags_retained':int(((values<q1-1.5*iqr)|(values>q3+1.5*iqr)).sum()),
        'deleted':0, 'imputed_numeric':0, 'winsorized':0, 'smoothed_observations':0}
pv=np.full((365,4,144),np.nan)
for day in range(365):
    for version in range(4):
        start=version*36
        anchor=(data['pv'][day,start-1]*6 if start else
            data['pv'][day-1,-1]*6 if day else 0.0)
        vals=np.r_[anchor,hourly[day,version]]
        edges=np.arange(145-start)/6
        powers=np.interp(edges,np.arange(25),vals)
        pv[day,version,start:]=(powers[:-1]+powers[1:])/12
    data.update(hourly_pv=hourly,pv_issued=pv,actual_price=prices)
audit={'pass':True,'checks':checks,'date_forward_fills_structural_only':filled,
    'dates':365,'unique_issues':1460,'issue_hours':[0,6,12,18],
    'forecast_horizons_hours':list(range(1,25)),
    'alignment':'Hourly lead endpoints; piecewise-linear integration over 10-minute cells; lead-zero
anchor from last completed observed PV interval; only current-day suffix traded.',
    'input_files':base_audit['input_files']+[
        {'path':name,'sha256':hashlib.sha256((root/name).read_bytes()).hexdigest()}
        for name in ['附件/附件 3.xlsx','附件/附件 4.xlsx']],
    'unusable_past_slots':'NaN intentionally marks targets before each issue; never supplied to a
solver',
    'auditor_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest()}
if audit_path:dump_json(audit_path,audit)
return data,audit

if __name__=='__main__':
    import argparse,json
    p=argparse.ArgumentParser();p.add_argument('--data-root',required=True)
    p.add_argument('--out',default='artifacts/q34/input_audit.json');args=p.parse_args()
    Path(args.out).parent.mkdir(parents=True,exist_ok=True)
    _,audit=read_extended(args.data_root,args.out)
    print(json.dumps(audit,ensure_ascii=False,indent=2))

```

src/v3_model.py

"""Exact LP reformulation and causal forecast-corrected dispatch.

Energy in kWh. Same settlement, time axis and realtime regulation as V1.
No training or tuning is performed here; inputs explicitly carry decisions.

```

"""
from dataclasses import dataclass, asdict
from functools import lru_cache
import time
import numpy as np
import pandas as pd
from scipy.optimize import linprog
from scipy.sparse import lil_matrix
from src.q12 import B, TOL, audit_trace, execute_day

```

```

@lru_cache(maxsize=32)
def structure(n, revision):
    size = (7 if revision else 5)*n+1
    a = lil_matrix(((3 if revision else 2)*n, size))
    for t in range(n):
        a[t,t], a[t,n+t], a[t,2*n+t], a[t,3*n+t] = 1,-1,1,-1
        a[n+t,4*n+t+1], a[n+t,4*n+t] = 1,-1
        a[n+t,n+t], a[n+t,2*n+t] = -B.eta_c,1/B.eta_d
        if revision:
            a[2*n+t,t], a[2*n+t,5*n+1+t], a[2*n+t,6*n+1+t] = 1,-1,1
    return a.tocsr()

def remove_cycles(charge, discharge, spill):
    """Preserve grid purchases and ALL SOC values, eliminate simultaneous flow.

    x=min(c,d/(eta_c*eta_d)); c'=c-x; d'=d-eta_c*eta_d*x;
    w'=w+(1-eta_c*eta_d)*x. Requires free unbounded curtailment.
    """
    rho=B.eta_c*B.eta_d
    x=np.minimum(charge,discharge/rho)
    return charge-x,discharge-rho*x,spill+(1-rho)*x

def solve(load,pv,price,initial,terminal=6000.,old=None,refund=True,equal=False):
    load,pv,price=map(lambda x:np.asarray(x,dtype=float),(load,pv,price))
    n=len(price);revision=old is not None;size=(7 if revision else 5)*n+1
    if not all(x.shape==(n,) and np.isfinite(x).all() for x in (load,pv,price)):
        raise ValueError('Nonfinite or misaligned dispatch inputs')
    if np.any(price<=0) or not B.minimum<=terminal<=B.maximum:
        raise ValueError('Positive prices and admissible terminal storage required')
    bounds=[(0,None)]*size
    bounds[n:3*n]=[(0,B.limit)]*(2*n)
    bounds[4*n:5*n+1]=[(B.minimum,B.maximum)]*(n+1)
    bounds[4*n]=(initial,initial);bounds[5*n]=(terminal,terminal if equal else B.maximum)
    cost=np.zeros(size)
    rhs=np.r_[load-pv,np.zeros(n)]
    if revision:
        old=np.asarray(old,dtype=float)
        assert old.shape==(n,) and np.isfinite(old).all() and old.min()>=-TOL
        cost[5*n+1:6*n+1]=1.5*price
        cost[6*n+1:]=(-.5 if refund else .5)*price
        bounds[6*n+1:]=[(0,max(float(v),0)) for v in old]
        rhs=np.r_[rhs,old]
    else:cost[:n]=price
    t=time.perf_counter()
    fit=linprog(cost,A_eq=structure(n,revision),b_eq=rhs,bounds=bounds,method='highs',
        options={'time_limit':30,'dual_feasibility_tolerance':1e-
8,'primal_feasibility_tolerance':1e-8})
    if not fit.success:raise RuntimeError(f'LP status={fit.status}: {fit.message}')
    x=fit.x;c,d,w=remove_cycles(x[n:2*n],x[2*n:3*n],x[3*n:4*n])
    f=pd.DataFrame(dict(plan_kwh=x[:n],charge_kwh=c,discharge_kwh=d,spill_kwh=w,
        soc_start_kwh=x[4*n:5*n],soc_end_kwh=x[4*n+1:5*n+1],load_kwh=load,pv_kwh=pv,
        emergency_kwh=np.zeros(n),price=price))
    physical=audit_trace(f,initial=initial)
    return f,dict(objective=float(fit.fun),seconds=time.perf_counter()-t,status=int(fit.status),
        iterations=int(fit.nit),physical=physical)

@dataclass(frozen=True)
class Policy:
    quantile:float=.7
    terminal:float=6000.
    mask:int=0
    beta:float=0.
    refund:bool=True
    lookback:int=18
    decay:int=36

def corrected_load(data,pred,beta,lookback=18,decay=36):
    """At each release, use only the just-completed <=3h load residuals.

    No observed PV is used to correct load; no current/future interval enters.

```

```

Correction decays with lead, preventing all-day propagation of a short shock.
"""
out=np.repeat(pred['load'][:,None,:],4,axis=1).copy()
for v in range(1,4):
    start=36*v;left=max(0,start-lookback)
    err=(data['load'][:,left:start]-pred['load'][:,left:start]).mean(axis=1)
    lead=np.arange(144-start)+.5
    out[:,v,start:]=np.maximum(0,out[:,v,start:]+beta*err[:,None]*np.exp(-lead[None,:]/decay))
return out

def risk_buffers(data,load_pred,pv_pred,q):
    out=np.zeros_like(load_pred)
    if q==0:return out
    for day in range(7,len(load_pred)):
        left=max(1,day-28)
        error=data['load'][left:day,None,:]-data['pv'][left:day,None,:]-load_pred[left:day]+pv_pred[left:day]
        for v in range(4):
            start=36*v
            for slot in range(start,144):
                out[day,v,slot]=max(0,float(np.quantile(error[:,v,max(start,slot-3):min(144,slot+4)],q)))
    return out

def run(data,load_pred,pv_pred,buffers,policy,*,start,stop,initial,name,prices=None,retain=True):
    state=float(initial);frames=[];days=[];ledger=[];solves=[]
    for day in range(start,stop):
        date=str(data['dates'][day].date());s0=state
        realized=data['price'] if prices is None else data['actual_price'][day]
        p0=data['price'] if prices is None else prices[day,0,:144]
        sol,meta=solve(load_pred[day,0]+buffers[day,0],pv_pred[day,0],p0,state,policy.terminal)
        solves.append({'date':date,'version':0,**{k:meta[k] for k in
['seconds','status','iterations','objective']})
        plan=np.maximum(sol.plan_kwh.to_numpy(),0);original=plan.copy()
        up=np.zeros(144);down=np.zeros(144);blocks=[]
        for v in range(4):
            first=v*36;last=first+36
            if v and policy.mask & (1<(v-1)):
                p=data['price'][first:] if prices is None else prices[day,v,:144-first]
                old=plan[first:].copy()
                sol,meta=solve(load_pred[day,v,first:]+buffers[day,v,first:],pv_pred[day,v,first:],
                    p,state,policy.terminal,original[first:],policy.refund)
                solves.append({'date':date,'version':v,**{k:meta[k] for k in
['seconds','status','iterations','objective']})
                new=np.maximum(sol.plan_kwh.to_numpy(),0);delta=new-old
                up[first:]+=np.maximum(delta,0);down[first:]+=np.maximum(-delta,0);plan[first:]=new
                if retain:
                    for j,slot in enumerate(range(first,144)):
                        ledger.append([date,v,first,slot,float(old[j]),float(new[j]),max(float(delta[j]),0),
                            max(-float(delta[j]),0),float(p[j]),float(load_pred[day,v,slot]),
                            float(pv_pred[day,v,slot]),float(buffers[day,v,slot]),state)])
        f=execute_day(plan[first:last],data['load'][day,first:last],data['pv'][day,first:last],state,realized[first:
last])
        state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)
        f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',np.arange(144))
        up=np.maximum(plan-original,0);down=np.maximum(original-plan,0)
        f['original_plan_kwh']=original;f['increase_kwh']=up;f['decrease_kwh']=down
        f['original_cost_yuan']=realized*original;f['increase_cost_yuan']=realized*1.5*up
        f['decrease_cost_yuan']=realized*(-.5 if policy.refund else .5)*down
        f['emergency_cost_yuan']=realized*5*f.emergency_kwh
    f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
    np.testing.assert_allclose(f.plan_kwh,original+up-down,atol=TOL,rtol=0)
    item={'date':date,'strategy':name,'soc_start_kwh':s0,'soc_end_kwh':state}
    for c in f:
        if c.endswith('_yuan') or c in ['original_plan_kwh','plan_kwh','increase_kwh','decrease_kwh',
            'emergency_kwh','spill_kwh','charge_kwh','discharge_kwh']:item[c]=float(f[c].sum())
        item['emergency_intervals']=int((f.emergency_kwh>TOL).sum());days.append(item)
    if retain:frames.append(f)
    daily=pd.DataFrame(days);trace=pd.concat(frames,ignore_index=True) if retain else None
    versions=pd.DataFrame(ledger,columns=['date','version','effective_slot','slot','old_kwh','new_kwh',
        'increase_kwh','decrease_kwh','forecast_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision
_soc_kwh'])

```

```

summary={'strategy':name,'policy':asdict(policy),'days':stop-
start,'initial_soc':initial,'final_soc':state,
'solver_seconds':sum(x['seconds'] for x in solves),'solver_calls':len(solves),
'physical':audit_trace(trace,initial=initial) if retain else None,
**{c:float(daily[c].sum()) for c in daily if c.endswith('_yuan') or c in
['emergency_kwh','spill_kwh']}}
return daily,trace,versions,summary,pd.DataFrame(solves)

```

src/v5_model.py

```

"""Past-only, lead-specific online load residual calibration (energy in kWh)."""
import numpy as np

```

```

def adaptive_load(data, midnight, window=28, shrink=.2):
    """Fit one nonnegative slope per release/lead-hour using completed days.

    x: last three observed hours' mean midnight-forecast error.
    y: error in the target hour, on historical days only.
    Ridge penalty = shrink * sum(x**2); no future-day normalization.
    Within each target hour the correction is constant. Midnight is unchanged.
    """
    truth=data['load']; count=len(midnight)
    out=np.repeat(midnight[:,None,:],4,axis=1); records=[]
    error=truth[:count]-midnight
    for day in range(14,count):
        left=max(7,day-window)
        for v in range(1,4):
            start=v*36
            x=error[left:day,start-18:start].mean(axis=1)
            current=float(error[day,start-18:start].mean())
            denom=float(x*x)*(1+shrink)
            for slot in range(start,144,6):
                y=error[left:day,slot:slot+6].mean(axis=1)
                beta=float(np.clip(x@y/denom,0,1.5)) if denom>1e-12 else 0.
                out[day,v,slot:slot+6]=np.maximum(0,midnight[day,slot:slot+6]+beta*current)
                records.append(dict(day=day,issue_hour=v*6,target_slot=slot,train_first_day=left,
                                    train_last_day=day-1,n_train=day-left,beta=beta,observed_residual_kwh=current))
    return out,records

```

src/v5b_value.py

```

"""Continuous convex piecewise-linear storage values; no SOC discretization."""
from dataclasses import dataclass
import numpy as np
import pandas as pd
from src.q12 import B,audit_trace

```

```

@dataclass
class Value:
    x:np.ndarray
    y:np.ndarray
    def at(self,s):return np.interp(s,self.x,self.y)
    @property
    def slopes(self):return np.diff(self.y)/np.diff(self.x)

```

```

def restrict(f,lo=B.minimum,hi=B.maximum):
    lo=max(lo,float(f.x[0]));hi=min(hi,float(f.x[-1]))
    assert lo<=hi+1e-7
    x=np.unique(np.r_[lo,f.x[(f.x>lo+1e-8)&(f.x<hi-1e-8)],hi])
    y=f.at(x)
    if len(x)>2:
        slopes=np.diff(y)/np.diff(x);keep=np.r_[True,abs(np.diff(slopes))>1e-8,True]
        x,y=x[keep],y[keep]
    return Value(x,y)

```

```

def convolve(f,h):
    """Convex infimal convolution by merging segment slopes and lengths."""
    lengths=np.r_[np.diff(f.x),np.diff(h.x)];slopes=np.r_[f.slopes,h.slopes]
    order=np.argsort(slopes,kind='stable');lengths=lengths[order];slopes=slopes[order]
    x=np.r_[f.x[0]+h.x[0],f.x[0]+h.x[0]+np.cumsum(lengths)]
    y=np.r_[f.y[0]+h.y[0],f.y[0]+h.y[0]+np.cumsum(lengths*slopes)]

```

```

return restrict(Value(x,y))

def purchase_stage(net,p):
    lo=-B.eta_c*B.limit;hi=B.limit/B.eta_d
    zero=net/B.eta_d if net>=0 else net*B.eta_c
    x=np.unique(np.r_[lo,hi,np.clip(0,lo,hi),np.clip(zero,lo,hi)])
    y=p*np.maximum(0,net+np.where(x>=0,-B.eta_d*x,-x/B.eta_c))
    return Value(x,y)

def deterministic(load,pv,price,initial=6000,terminal=6000):
    n=len(price);values=[None]*(n+1);values[n]=Value(np.array([float(terminal)]),np.array([0.]))
    stages=[purchase_stage(l-v,p) for l,v,p in zip(load,pv,price)]
    for t in range(n-1,-1,-1):values[t]=convolve(values[t+1],stages[t])
    s=float(initial);records=[]
    for t,h in enumerate(stages):
        future=values[t+1];lo=max(future.x[0],s-h.x[-1]);hi=min(future.x[-1],s-h.x[0])
        choices=np.unique(np.r_[lo,hi,future.x,s-h.x],lo,hi))
        costs=future.at(choices)+h.at(s-choices);end=float(choices[np.argmin(costs)])
        c=max(0,(end-s)/B.eta_c);d=max(0,(s-end)*B.eta_d);g=max(0,load[t]-pv[t]+c-d);w=g+pv[t]+d-load[t]-c
        records.append([g,c,d,max(0,w),s,end,load[t],pv[t],0.,price[t]]);s=end

f=pd.DataFrame(records,columns=['plan_kwh','charge_kwh','discharge_kwh','spill_kwh','soc_start_kwh','soc_end_kwh',
'_kwh','load_kwh','pv_kwh','emergency_kwh','price'])
audit_trace(f,initial=initial)
assert abs(float(price@f.plan_kwh)-float(values[0].at(initial)))<1e-6
return f,values

def recourse_values(net,price,terminal_rate):
    """A historical path's perfect-information value; used only as an approximation.

    Emergency supply serves deficits only; it cannot charge storage. Surplus
    charges greedily, which is optimal for these nonincreasing future values.
    """
    n=len(net);out=[None]*(n+1)
    out[n]=Value(np.array([B.minimum,6000.,B.maximum]),terminal_rate*np.array([4800.,0.,0.]))
    for t in range(n-1,-1,-1):
        if net[t]>0:
            top=min(float(net[t]),B.limit)/B.eta_d
            h=Value(np.array([0.,top]),5*price[t]*np.array([net[t],net[t]-B.eta_d*top]))
            out[t]=convolve(out[t+1],h)
        else:
            a=min(-float(net[t]),B.limit)*B.eta_c;f=out[t+1]
            x=np.unique(np.r_[B.minimum,np.clip(f.x-a,B.minimum,B.maximum),B.maximum])
            out[t]=restrict(Value(x,f.at(np.minimum(B.maximum,x+a))))
    return out

def execute_value(plan,load,pv,initial,price,paths,terminal_rate):
    all_values=[recourse_values(net,price,terminal_rate) for net in paths]
    s=float(initial);rows=[]
    for t,(g,l,v,p) in enumerate(zip(plan,load,pv,price)):
        net=float(l-v-g);c=d=e=w=0.
        if net<=0:
            c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-min(net,B.limit)/B.eta_d);hi=s
            choices=np.unique(np.r_[lo,hi,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<hi)] for f in all_values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in all_values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end

f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','spill_kwh',
'soc_start_kwh','soc_end_kwh','price'])
audit_trace(f,initial=initial);return f

```

src/v7_price.py

```

"""Causal seasonal and ridge price forecasts from observed history."""
from pathlib import Path
import argparse
import json
import numpy as np
from src.q12 import ridge_fit

```

```

from src.q34_data import read_extended

def features_at(flat, origin):
    target = np.arange(origin, origin + 144)
    assert (target - 144 < origin).all() and origin >= 1008
    hour = (target % 144 + .5) / 144
    dow = (target // 144 + 2) % 7
    return np.column_stack([flat[target-144], flat[target-1008],
        np.sin(2*np.pi*hour), np.cos(2*np.pi*hour),
        np.sin(2*np.pi*dow/7), np.cos(2*np.pi*dow/7)]).astype('float32')

def ridge_features(x):
    a, b = x[..., 0], x[..., 1]
    context = np.stack([a.mean(-1), b.mean(-1), a.std(-1), b.std(-1)], axis=-1)
    context = np.broadcast_to(context[..., None, :], (*a.shape, 4))
    return np.concatenate([x, np.stack([a-b, a*b], axis=-1), context], axis=-1)

def reconstruct(data):
    flat = data['actual_price'].ravel()
    result = {k: np.full((365, 4, 144), np.nan) for k in ['seasonal', 'ridge']}
    for day in range(7, 365):
        for version in range(4):
            result['seasonal'][day, version] = features_at(flat, day*144+version*36)[: , :2].mean(-1)
    fits = [d for d in range(14, 365) if d in [14, 21] or data['dates'][d].day == 1]
    origins = np.arange(1008, len(flat)-143, 36)
    for i, day in enumerate(fits):
        end = fits[i+1] if i+1 < len(fits) else 365
        oo = origins[(origins+144 <= day*144) & (origins >= max(1008, day*144-60*144))]
        x = np.stack([features_at(flat, o) for o in oo])
        y = np.stack([flat[o:o+144] for o in oo]).astype('float32')
        mean, scale, coef, intercept = ridge_fit(ridge_features(x).reshape(-1, 12), y.ravel(), 1.)
        future = np.stack([features_at(flat, d*144+v*36) for d in range(day, end) for v in range(4)])
        values = ((ridge_features(future)-mean)/scale) @ coef + intercept
        result['ridge'][day:end] = np.maximum(values.reshape(end-day, 4, 144), .001)
    result['ridge'][7:14] = result['seasonal'][7:14]
    return result, fits

def residual_bands(actual, predictions, version=0):
    """Historical 10%-90% residual ranges, not guaranteed coverage intervals."""
    lower = np.full_like(actual, np.nan); upper = lower.copy()
    start = version*36
    for day in range(31, 365):
        left = max(7, day-28)
        residual = actual[left:day, start:] - predictions[left:day, version, :144-start]
        q = np.quantile(residual, [.1, .9], axis=0)
        point = predictions[day, version, :144-start]
        lower[day, start:] = np.maximum(.001, point + q[0])
        upper[day, start:] = np.maximum(.001, point + q[1])
    return lower, upper

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-root', required=True)
    parser.add_argument('--out', type=Path, default=Path('artifacts/v7'))
    args = parser.parse_args(); args.out.mkdir(parents=True, exist_ok=True)
    data, _ = read_extended(args.data_root); result, fits = reconstruct(data)
    prior = np.load('artifacts/q34/price_forecasts.npz')
    errors = {}
    for key in result:
        errors[key] = float(np.max(np.abs(result[key][14:] - prior[key][14:])))
    np.testing.assert_allclose(result[key][14:], prior[key][14:], atol=1e-10, rtol=0)
    np.savez_compressed(args.out/'price_forecasts.npz', **result)
    audit = dict(status='PASS', ridge_refits=len(fits), max_cache_difference=errors,
        information='All feature prices and training targets precede issue time',
        models=['seasonal', 'ridge'], alpha=1., training_days=60)
    (args.out/'price_audit.json').write_text(json.dumps(audit, ensure_ascii=False, indent=2), encoding='utf-8')

```

```

print(json.dumps(audit, ensure_ascii=False))

if __name__ == '__main__':
    main()

src/v7_dispatch.py

"""State-value control in the remaining three cases, with causal prices."""
from pathlib import Path
import argparse,json,time
import numpy as np
import pandas as pd
from src.q12 import B,forecasts,audit_trace,execute_day,dump_json
from src.q34_data import read_extended
from src.v3_model import solve,risk_buffers
from src.v5_model import adaptive_load
from src.v5b_value import recourse_values

OUT=Path('artifacts/v7/replay')
def execute_prefix(plan,load,pv,initial,actual_price,forecast_price,paths,steps,release_aware=False):
    """Future values use forecast prices; only executed slots read real prices."""
    continuation_price=np.array(forecast_price,copy=True)
    if release_aware:continuation_price[steps:]*=.3
    values=[recourse_values(path,continuation_price,0.) for path in paths];s=float(initial);rows=[]
    for t in range(steps):
        g,l,v,p=plan[t],load[t],pv[t],actual_price[t];net=float(1-v-g);c=d=e=w=0.
        if net<=0:c=min(-net,B.limit,(B.maximum-s)/B.eta_c);w=-net-c;end=s+B.eta_c*c
        else:
            lo=max(B.minimum,s-
min(net,B.limit)/B.eta_d);choices=np.unique(np.r_[lo,s,*[f[t+1].x[(f[t+1].x>lo)&(f[t+1].x<s)] for f in
values]])
            costs=5*p*(net-B.eta_d*(s-choices))+np.mean([f[t+1].at(choices) for f in values],axis=0)
            end=float(choices[np.argmin(costs)]);d=B.eta_d*(s-end);e=max(0,net-d)
            rows.append([g,l,v,c,d,e,w,s,end,p]);s=end
f=pd.DataFrame(rows,columns=['plan_kwh','load_kwh','pv_kwh','charge_kwh','discharge_kwh','emergency_kwh','sp
ill_kwh','soc_start_kwh','soc_end_kwh','price'])
    audit_trace(f,initial=initial);return f

def main():
    ap=argparse.ArgumentParser();ap.add_argument('--data-root',required=True);ap.add_argument('--
case',choices=['q3','q42','q43'],required=True);ap.add_argument('--extra-
only',action='store_true');a=ap.parse_args()
    OUT.mkdir(parents=True,exist_ok=True)
    case=a.case;rolling=case!='q42';q=.5 if rolling else .7
    controls=['greedy','value','value_release'] if rolling else ['greedy','value']
    dump_json(OUT/f'{case}_design.json',dict(candidates=controls,selection='January cost; greedy wins ties',
    quantile=q,terminal=6000,scenario_days=28,terminal_value_rate=0.,
    load_selector='Retain earlier V5 causal monthly load selections as a shared forecast input',
    price_information='Only the executed slot sees actual prices; future value uses current-release
price forecast',
    release_extension='Added after January frozen-contract value performed poorly; assumes future
shortage can enter a 1.5p adjusted contract after next issue. Actual bills unchanged.'))

data,_=read_extended(a.data_root);pred,_=forecasts(data,'ridge',(1.,1.));base=np.repeat(pred['load'][:,None,
:],4,axis=1)
    loads={'baseline':base,'adaptive':adaptive_load(data,pred['load'])[0]}
    pv=data['pv_issued'] if rolling else np.repeat(pred['pv'][:,None,:],4,axis=1)
    rr={k:risk_buffers(data,ll,pv,q) for k,ll in loads.items()}
    pp=None if case=='q3' else np.load('artifacts/v7/price_forecasts.npz')['ridge' if case=='q42' else
'seasonal']
    selections=pd.read_csv('artifacts/v5/online_selection.csv');mode=3 if case=='q3' else 4
    choices={int(r.month):r.candidate for r in
selections[(selections['mode']==mode)&selections.selected].itertuples())
    def run(control,start,stop,retain):
        state=6244.25689138887 if start==21 else
9902.811287870369;frames=[];daily=[];versions=[];tick=time.perf_counter()
        for day in range(start,stop):
            date=str(data['dates'][day].date());month=int(data['dates'][day].month);name=choices.get(month,'baseline')
            if rolling else 'baseline'
                ll=loads[name];buffer=rr[name];p0=data['price'] if pp is None else
pp[day,0,:144];actual=data['price'] if pp is None else data['actual_price'][day]
nominal,_=solve(ll[day,0]+buffer[day,0],pv[day,0],p0,state);original=nominal.plan_kwh.to_numpy().copy();plan
=original.copy();blocks=[]

```

```

        for ver in (range(4) if rolling else [0]):
            first=ver*36;steps=36 if rolling else 144;p=data['price'][first:] if pp is None else
pp[day,ver,:144-first]
            if ver:
                old=plan[first:].copy();nominal,_=solve(ll[day,ver,first:]+buffer[day,ver,first:],pv[day,ver,first:],p,state
,old=original[first:]);plan[first:]=np.maximum(nominal.plan_kwh,0)
                if retain:
                    for j,t in
enumerate(range(first,144)):versions.append([date,ver,first,t,float(old[j]),float(plan[t]),float(p[j]),float
(ll[day,ver,t]),float(pv[day,ver,t]),float(buffer[day,ver,t]),state))
                    if
control=='greedy':f=execute_day(plan[first:first+steps],data['load'][day,first:first+steps],data['pv'][day,f
irst:first+steps],state,actual[first:first+steps])
                    else:
                        left=max(7,day-28)
                        L=np.maximum(0,ll[day,ver,first:]+data['load'][left:day,first:]-ll[left:day,ver,first:])
                        V=np.maximum(0,pv[day,ver,first:]+data['pv'][left:day,first:]-pv[left:day,ver,first:])
                        f=execute_prefix(plan[first:],data['load'][day,first:],data['pv'][day,first:],state,actual[first:],p,L-V-
plan[first:],steps,control=='value_release')
                        state=float(f.soc_end_kwh.iloc[-1]);blocks.append(f)

f=pd.concat(blocks,ignore_index=True);f.insert(0,'date',date);f.insert(1,'slot',range(144));f['original_plan
_kwh']=original
        f['increase_kwh']=np.maximum(plan-original,0);f['decrease_kwh']=np.maximum(original-plan,0)

f['original_cost_yuan']=actual*original;f['increase_cost_yuan']=1.5*actual*f.increase_kwh;f['decrease_cost_y
uan']=-.5*actual*f.decrease_kwh;f['emergency_cost_yuan']=5*actual*f.emergency_kwh

f['total_cost_yuan']=f[['original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan'
]].sum(axis=1)
        daily.append(dict(date=date,soc_start_kwh=float(f.soc_start_kwh.iloc[0]),soc_end_kwh=state,
**{k:float(f[k].sum()) for k in
['total_cost_yuan','original_cost_yuan','increase_cost_yuan','decrease_cost_yuan','emergency_cost_yuan','pla
n_kwh','emergency_kwh','charge_kwh','discharge_kwh','spill_kwh']}))
        if retain:frames.append(f)
        if retain and data['dates'][day].day==1:print(case,control,date,flush=True)

d=pd.DataFrame(daily);s=dict(case=case,control=control,cost=float(d.total_cost_yuan.sum()),final_soc=state,e
mergency_kwh=float(d.emergency_kwh.sum()),seconds=time.perf_counter()-tick)
        if retain:
            f=pd.concat(frames,ignore_index=True);s['physical']=audit_trace(f,initial=9902.811287870369);d.to_csv(OUT/f'
{case}_{control}_daily.csv',index=False,float_format='%.17g');f.to_csv(OUT/f'{case}_{control}_intervals.csv.
gz',index=False,float_format='%.17g')
            if
versions=pd.DataFrame(versions,columns=['date','version','effective_slot','slot','old_kwh','new_kwh','foreca
st_price','forecast_load_kwh','forecast_pv_kwh','reserve_kwh','decision_soc_kwh']).to_csv(OUT/f'{case}_{cont
rol}_versions.csv.gz',index=False,float_format='%.17g')
            return s
        prior=json.loads((OUT/f'{case}_validation.json').read_text(encoding='utf-8'))['validation'] if
a.extra_only else []
        selected_controls=['value_release'] if a.extra_only else controls
        val=prior+[run(c,21,31,False) for c in selected_controls];chosen=min(val,key=lambda
x:(x['cost'],x['control']!='greedy'))['control']

dump_json(OUT/f'{case}_validation.json',{'validation':val,'selected':chosen});print(case,'January',val,chose
n,flush=True)
        prior=json.loads((OUT/f'{case}_summary.json').read_text(encoding='utf-8')) if a.extra_only else []
        results=prior+[run(c,31,365,True) for c in
selected_controls];dump_json(OUT/f'{case}_summary.json',results);print(case,results,flush=True)
if __name__=='__main__':main()

```